

领域	工具和技术		知识和技能	
价值驱动的工作交付	• ROI、NPV、IRR • 敏捷挣值管理 • 产品路线图 • 价值流映射 • 限制WIP • 相对排序	• 风险调整待办事项 • 累积图 • 任务、看板 • 章程 • 客户价值排序 • 风险燃尽图	• 原型、模拟、演示 • 增量交付 • 优先级排序 • 项目和质量标准	• 敏捷合同 • 敏捷结算 • 系统思考 • 敏捷方法的差异 • 价值分析
干系人参与	• 线框图 • 仆人式领导 • 用户故事、待办事项 • 冲突解决 • 敏捷建模 • 速率	• 信息辐射源 • 分布式团队 • 人物创建 • 燃起图、燃尽图 • 用户故事地图 • 协商	• 干系人价值 • 沟通管理 • 领导技术和工具 • 干系人管理 • 积极倾听	• 引导方法 • 全球化、文化、团队差异 • 供应商管理 • 参与决策模型
提升团队绩效	• 每日站会 • 集中办公 • 团队空间 • 敏捷工具	• 适应型领导 • 情商	• 头脑风暴 • 团队授权 • 教练与指导	• 高绩效团队 • 团队激励 • 集中办公和分布式办公
适应型规划	• 过程剪裁 • 发布和迭代规划 • 宽带德尔菲 • 规划扑克 • 渐进明细 • 时间盒	• 最小可售功能 • 理想时间 • 亲和估算 • 相对大小 • 故事点	• 进度估算 • 成本估算 • 预算估算 • 基于价值的解聚和排序	• 敏捷章程 • 商业案例分析 • 创新游戏
问题探测与解决	• 循环时间 • 缺陷泄露 • 持续集成 • 风险探测 • 频繁检查与验证	• 测试驱动开发、测试先行开发 • 验收测试驱动开发	• 问题解决 • 控制界限 • 失效模式	• 差异及趋势分析
持续改进	• 回顾		• 知识分享 • 过程分析 • 新的敏捷实践	• PMI道德准则 • 持续提升 • 自我评估

PMI-ACP与PMP相通的地方

- 商业论证
- 干系人管理
- 章程
- 挣值
- 质量
- 风险管理
- 持续改进/改善等
- 软技能方面的知识

考试要点

- 敏捷知识与实践
- 敏捷术语与实践操作
- 情景分析与敏捷实践
- 常用敏捷框架
- PMI考试风格

课程组成

敏捷
基础

敏捷价
值论证

打造敏
捷团队

敏捷框架

敏捷出现的背景

背景:从软件作坊到软件危机，再到软件工程，传统的软件工程理论使得软件系统和开发流程变得越来越复杂，导致:

- 过长的开发周期
- 无法应对迅速变化的市场环境，需求很难修改
- 超预算开发
- 项目进度检查困难
- 最终交付的软件，不一定能满足用户需求
- 人员流动频繁，官僚制度严重

实际操作时的问题

- 一方面市场的需求瞬息万变，很难实现产品需求的明确且完整地收集
- 一方面技术的发展也日新月异，对于所定义功能的可实现性也面临着多重不确定性的因素

敏捷基础

敏捷是什么

- 敏捷**不是**一个流程或者方法
- 敏捷**不是**一套固定模板
- 敏捷**不是**一种可以直接重用的方法
- 敏捷是一种为了适应不断变化的需求，以人为核心，通过迭代增量等方式的开发方法。其本质是一种实践。

敏捷的三根支柱

- 敏捷是基于「**经验型**」的方法，并不依赖于「**预测**」，其成功依赖于三根支柱：
 - 透明性：信息透明
 - 经常性检查：获得反馈
 - 适应：根据反馈快速调整并作出改变

瀑布 VS 敏捷

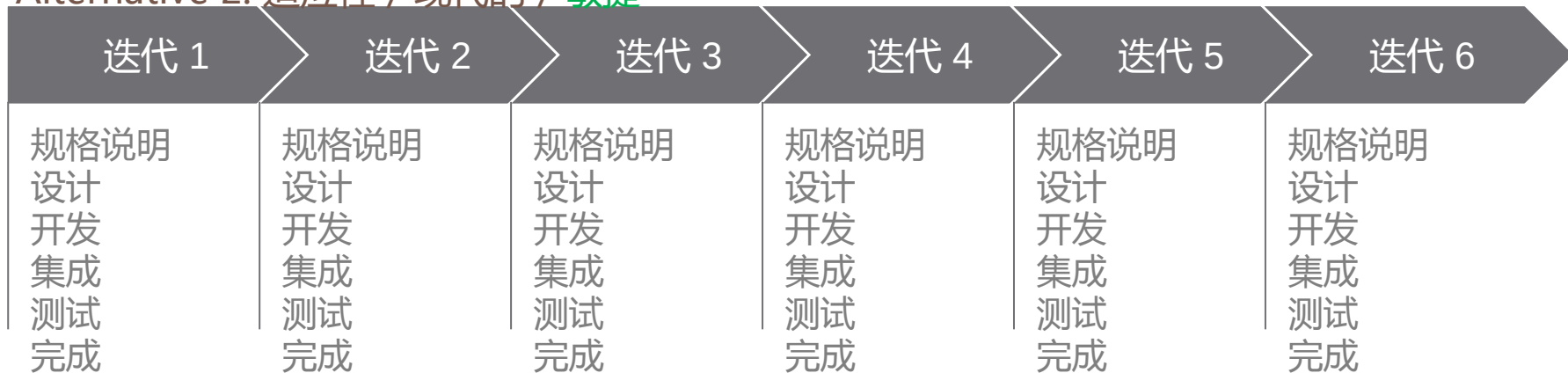
	传统项目管理	敏捷项目管理
流程	顺序、线性	迭代、并行
需求与范围	相对明确	随时变化
计划	完善的计划	渐进明细
流程文档	详尽的	足够的
交付	一次性交付	持续性交付
管理风格	领导式	仆人式

瀑布 VS 敏捷

Alternative 1: 预言式 / 传统的 / 瀑布式



Alternative 2: 适应性 / 现代的 / 敏捷



Waterfall VS Agile

Alternative 1: 预言式 / 传统的 / 瀑布式



可用软件

Alternative 2: 适应性 / 现代的 / 敏捷



可用软件

可用软件

可用软件

可用软件

可用软件

可用软件

Waterfall VS Agile

Alternative 1: 预言式 / 传统的 / 瀑布式



范围定义

Alternative 2: 适应性 / 现代的 / 敏捷



范围定义

范围定义

范围定义

范围定义

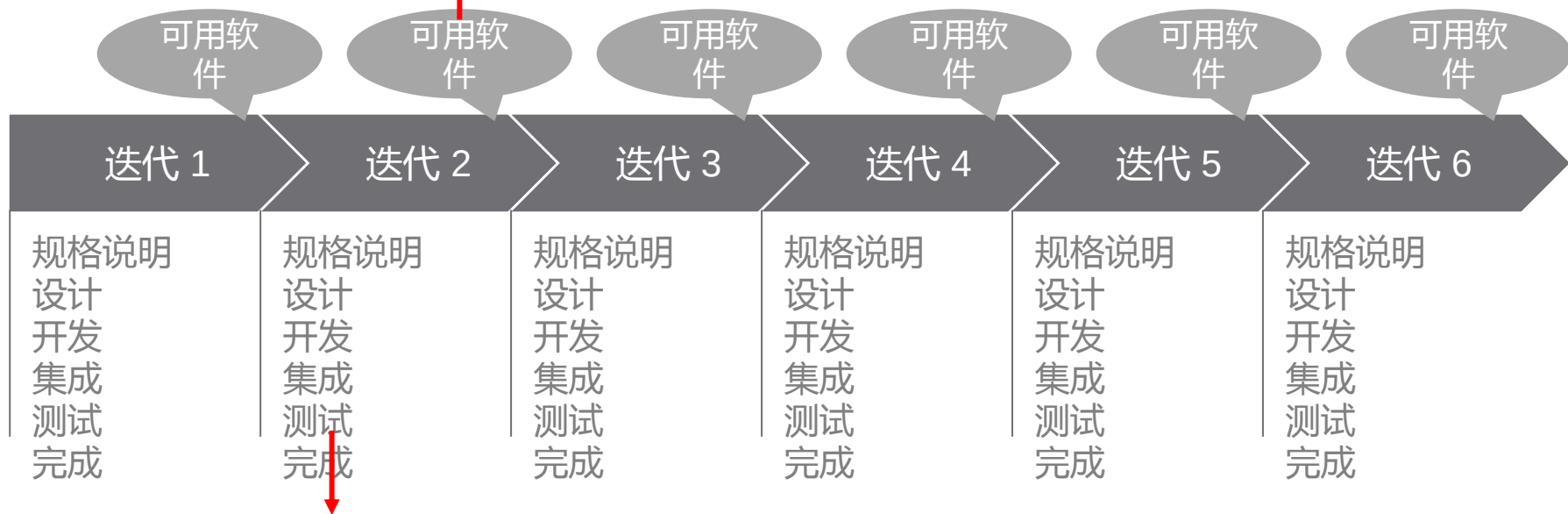
范围定义

范围定义

Agility敏捷

① Incremental增量

逐步交付，而不是一次性交付



② Iterative迭代

重复出现各个过程，而不是只出现一次

Iterative

1



2



3



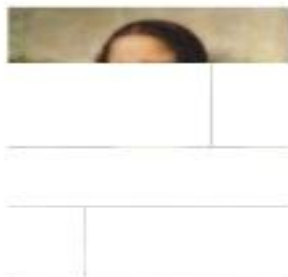
4



5

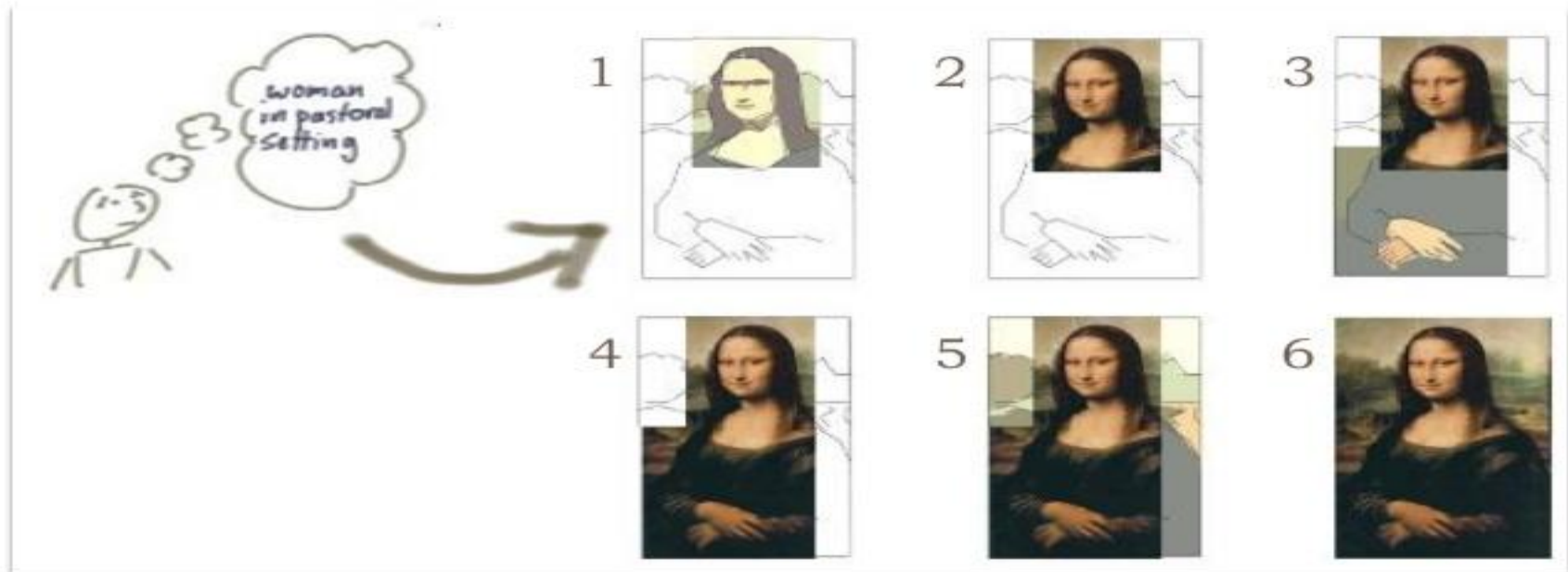


Incremental

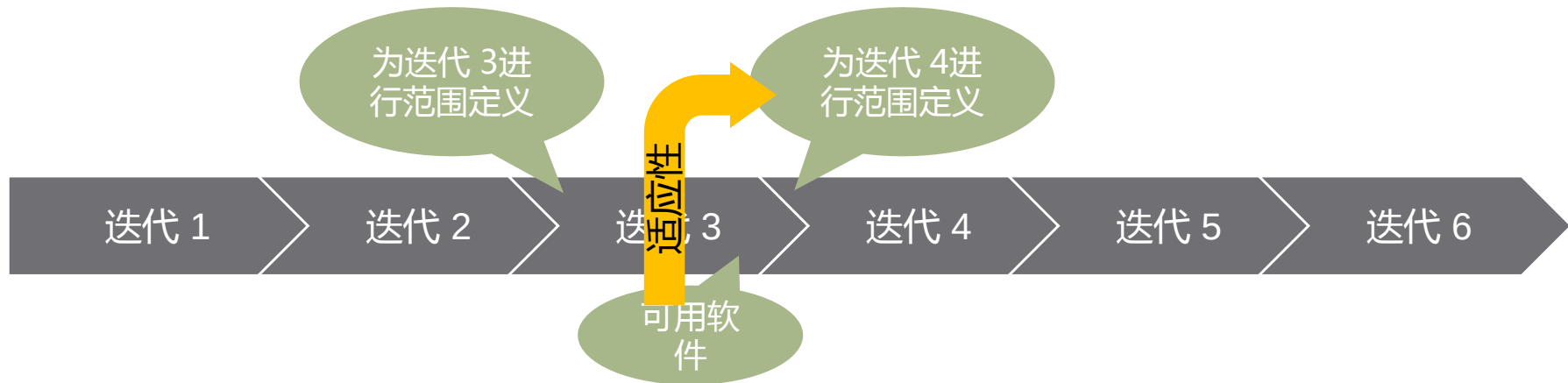


Outcome: Explain the difference between iterative and incremental and how that relates to User Story

Incremental and Iterative



Agility敏捷



适应性 ▶ 敏捷

敏捷软件开发宣言

我们一直在实践中探寻**更好**的软件开发方法，
身体力行的同时也帮助他人。由此我们建立了如下价值观：

个体和互动

高于

流程和工具

可用的软件

高于

详尽的文档

客户合作

高于

合同谈判

响应变化

高于

遵循计划

也就是说，尽管右项有其价值，
我们**更重视**左项的价值。

Principle 1

我们最重要的目标，是通过持续不断地尽早交付有价值的软件使客户满意。

Principle 2

欣然面对需求变更，即使在开发后期也一样。

为了客户的竞争优势，敏捷过程掌控变化。

Principle 3

经常地交付可工作的软件，相隔几星期或一两个月，
倾向于采取较短的周期。

Principle 4

业务人员和开发人员必须相互合作，一起工作，项目中的每一天都不例外。

Principle 5

激发个体的斗志，以他们为核心搭建项目。

提供所需的环境和支援，辅以信任，从而达成目标。

Principle 6

不论团队内外，传递信息效果最好效率也最高的方式是面对面的交谈。

敏捷

敏捷原则

Principle 7

可工作的软件是进度的首要度量标准。

Principle 8

敏捷过程倡导可持续开发。

责任人、开发人员和用户要能够共同维持其步调稳定延续。

Principle 9

坚持不懈地追求技术卓越和良好设计，敏捷能力由此增强。

Principle 10

以简洁为本，它是极力减少不必要工作量的艺术。

Principle 11

最好的架构、需求和设计出自自组织团队。

Principle 12

团队定期地反思如何能提高成效，并依此调整自身的举止表现。

Minimally Marketable Feature

- Minimum: Requires a mutual agreement of DoD
- Feature: Something that is perceived as value by user
- Marketable: Provides significant value to the customer

客户价值排序

- Simple Scheme
- MoSCoW方法
- 100 - Point Method
- Kano 分析

客户价值排序 - Simple Scheme

- 简单优先级排序——直接设定优先级，比如优先级1，2，3
- 前提：对每个优先级的**定义明确**的优先级说明与定义，否则会出现全部是高优先级的列表

客户价值排序 - MoSCoW方法

- Must Have：必须有，基础性功能，缺少会让系统不可用/无价值
- Should Have：应该有，重要功能，使系统可以正常使用；如果缺乏会导致系统使用成本过高
- Could Have：可以有，系统增值功能
- Would Have But Not This Time：这次没有，其实是永远都没有

客户价值排序 - 100-Points Method

- 经常用在用例（ User Case ）为代表的功能排序
- 做法：
 - 每个干系人分配100点
 - 用100点对其认为重要的功能投票
 - 100点可以任意使用，不做任何限制

用户需求价值排序 - Kano Model

- 由Noriaki Kano开发（上世纪80年代）
- Kano模型定义了三个层次的需求
 - 基本型需求（基本因素）
 - 期望型需求（绩效因素）
 - 兴奋型需求（激励因素）
- 这是一种对客户需求和绩效指标分类的定性分析模型
- 理解客户需求和满意度的关系，有助于建立提高客户满意度的发布计划

风险管理

- 同PMBOK对风险的管理
- 将风险事项当作需求（负向价值需求），将可以**主动**应对的风险与正常需求同时进行排序
- 风险处理顺序
 - 高风险高价值：**优先处理**
 - 低风险高价值：其次处理
 - 低风险低价值：最后处理
 - 高风险低价值：不予处理

敏捷合同

- 金钱无用&自由变更
- 分级固定价格合同
- 固定价值工作包
- 定制化合同

打造敏捷团队

打造敏捷团队

- 自组织团队
- 冲突解决
- 软技能
- 分布式团队

自组织团队 - 建立自组织团队

- 敏捷特征适合建立自组织团队

- ☐ 目标明确
- ☐ 共同达到目标
- ☐ 每个人天赋容易被激发

- 作法

- ☐ 忘记Title，接受Role
- ☐ 给予明确目标
- ☐ 反馈机制
- ☐ 团队协作
- ☐ 授权：计划、估算、对项目全权负责
- ☐ 提升团队主观能动性

自组织团队 - 特征

- 分散式管理，扁平化组织
- 自适应，不断适应改变的环境，自我调整
- 面对面沟通
- 反馈环
- 具有弹性

自组织团队 - 环境因素影响

- 信息：提供数据的团队，其成员需要胜任计划和执行的工作
- 基础设备：合适的物理空间、技术设备和金钱
- 教育：团队需要培训、教练和技术支援
- 奖励：以正向反馈刺激团队

自组织团队 - 规模

- 5 - 9人：规模小到足够敏捷，同时大到能完成复杂工作
- 少于3个人，团队交互较少，生产力增长有限，且会受限于人员技能限制，导致无法正常交付增量产品
- 大于9人，沟通途径($n*(n-1)/2$)过大，沟通成本过高
- 产品负责人和敏捷教练不在人数内

$$5 \leq \text{人数} \leq 9$$

分布式团队

- 在全球化的环境，很多项目都存在于文化多样性的环境中。理解并利用**文化差异**，了解各自的**沟通偏好和风格**，更有可能创建一个互相信任和共赢的氛围
- 非常有效的一个方式是组织面对面(face-to-face)的kick-off meeting，甚至有条件的包括kick-off events(如每个迭代开始时)。
- 其他方式:
 - 借调、临时分配等方式初始不同地区的同时有机会一起工作
 - XP中的结对编程，有利于从他人身上学习，了解别人沟通方式等
 - 组织回顾会议 – 需要特别考虑文化差异
- 安排地理上分散的团队在早期在一起工作，有利于后期的有效沟通

适应性计划

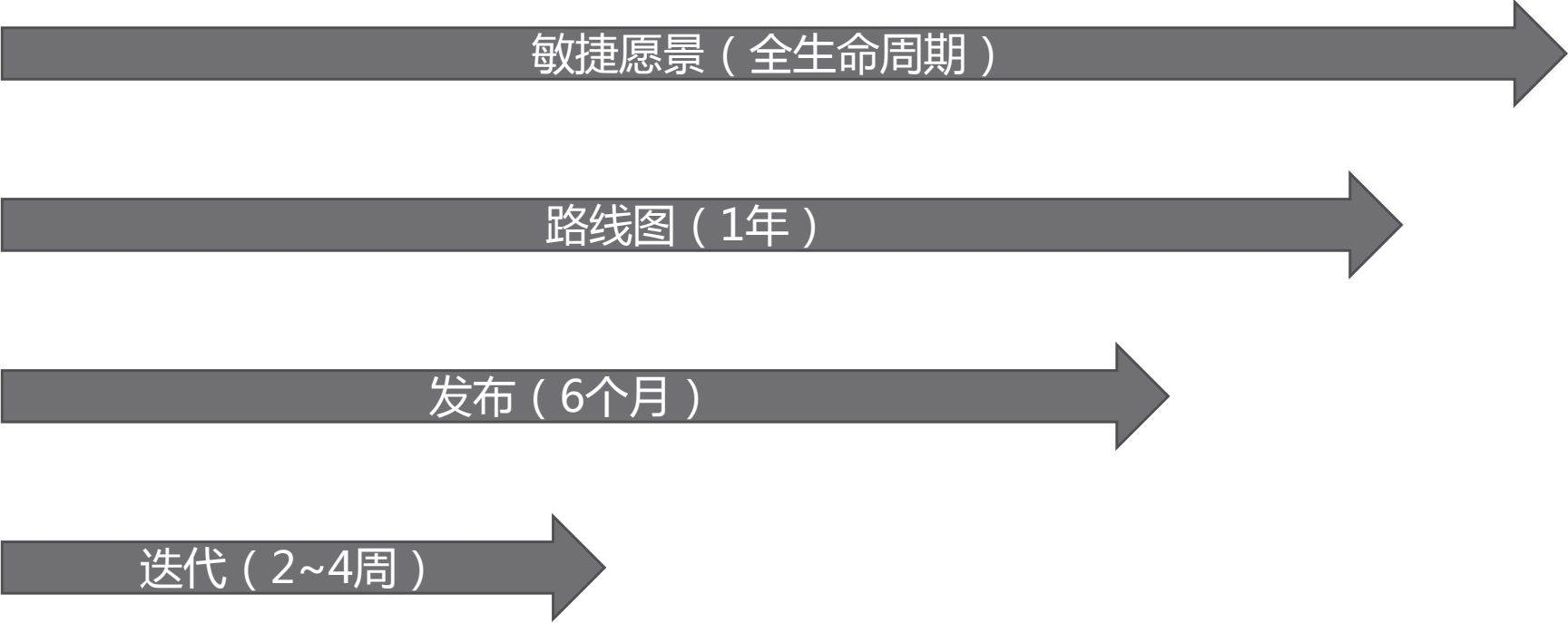
适应性计划的必要性

- 项目的不确定性
- 敏捷项目是以价值为驱动
- 风险需要降低
- 环境不断的变化：市场、技术、外部因素

敏捷计划 VS 瀑布计划

1. 敏捷关注真实需求，体现出渐进明细
2. 敏捷计划关注于整个项目生命周期
3. 敏捷计划需要及时对中期计划进行调整

敏捷计划的层次



敏捷愿景（全生命周期）

路线图（1年）

发布（6个月）

迭代（2~4周）

敏捷计划层次-敏捷愿景

- 意义

- ☐ 第一个文档
- ☐ 制订了目标和战略
- ☐ 可以作为项目章程

- 愿景核心

- ☐ Why
- ☐ What
- ☐ When
- ☐ Who
- ☐ Where
- ☐ How

基于价值的分析-定义愿景

- 设计产品盒子

- 4~6人一个小组

- 正面和背面

- 正面：名称、标志、关键卖点

- 背面：详细功能描述和操作要求

- 电梯测试说明书

- 快速讲清楚自己的要求

- 目标客户

- 解决的问题

- 产品优点

- 与竞品的区别

敏捷计划层次-产品路线图

- 定义：产品负责人拥有，是产品需求高层次概述，常用作特性优先级排序、特性归类和**粗略**时间框架确定的工具。

	2014				2015			
	1Q	2Q	3Q	4Q	1Q	2Q	3Q	4Q
Premium 4G			MT6595 LTE SoC LTE, Octa, 2.2-2.5GHz, big LITTLE, 28nm HPM, 4 A7+4 A57	MT6795 LTE SoC; 64 bit LTE, Octa, 2.2GHz, 28nm HPM, 4 A57+4 A53				MT67XX LTE Cat 1 w/CA
Mainstream 4G		MT6290+ MT6592/M Octa core, 28nm, 8x A7, 1.7-2.0GHz/ 1.4GHz, HD1080/HD720, 16MP		MT6752 LTE SoC; 64 bit LTE, Octa, 1.7GHz, 28nm HPM, 8 A53		MT6753 LTE + C2K SoC Octa, 1.5GHz, 8 A53, 6 mode with CDMA, 28nm		
Entry 4G		MT6290+ MT6592/M 28nm, 4x Cortex A7, 1.3GHz, qHD, 13MP		MT6732 LTE SoC; 64 bit LTE, Quad, 1.5GHz, 28nm, 4 A53		MT6735 LTE + C2K SoC Quad A53, 1.5GHz, 13MP, with CDMA, 28nm LP MT6735M LTE + C2K SoC Quad A53, 1.0GHz, 8MP, CDMA, 28nm LP		
Entry 3G	28nm, 4x Cortex A7, 1.3GHz, 960x540	MT6582M MT6572/M 28nm, 2x Cortex A7, 1.2GHz, 960x540	MT6571 28nm, 1.0 GHz, 2x Cortex A7, PowerVR, W+TD				MT6580 Quad A7, 1.3GHz, cost down MT6570 Dual A7, 1.3GHz cost down	

敏捷计划层次-产品路线图

四个步骤画出产品路线图：

1. 确认需求——明确后续待办事项
2. 需求分类——分出不同主题（ Theme ）
3. 评估相对工作量——计划扑克、理想时长
4. 评估**粗略**时间框架

敏捷计划层次-发布计划

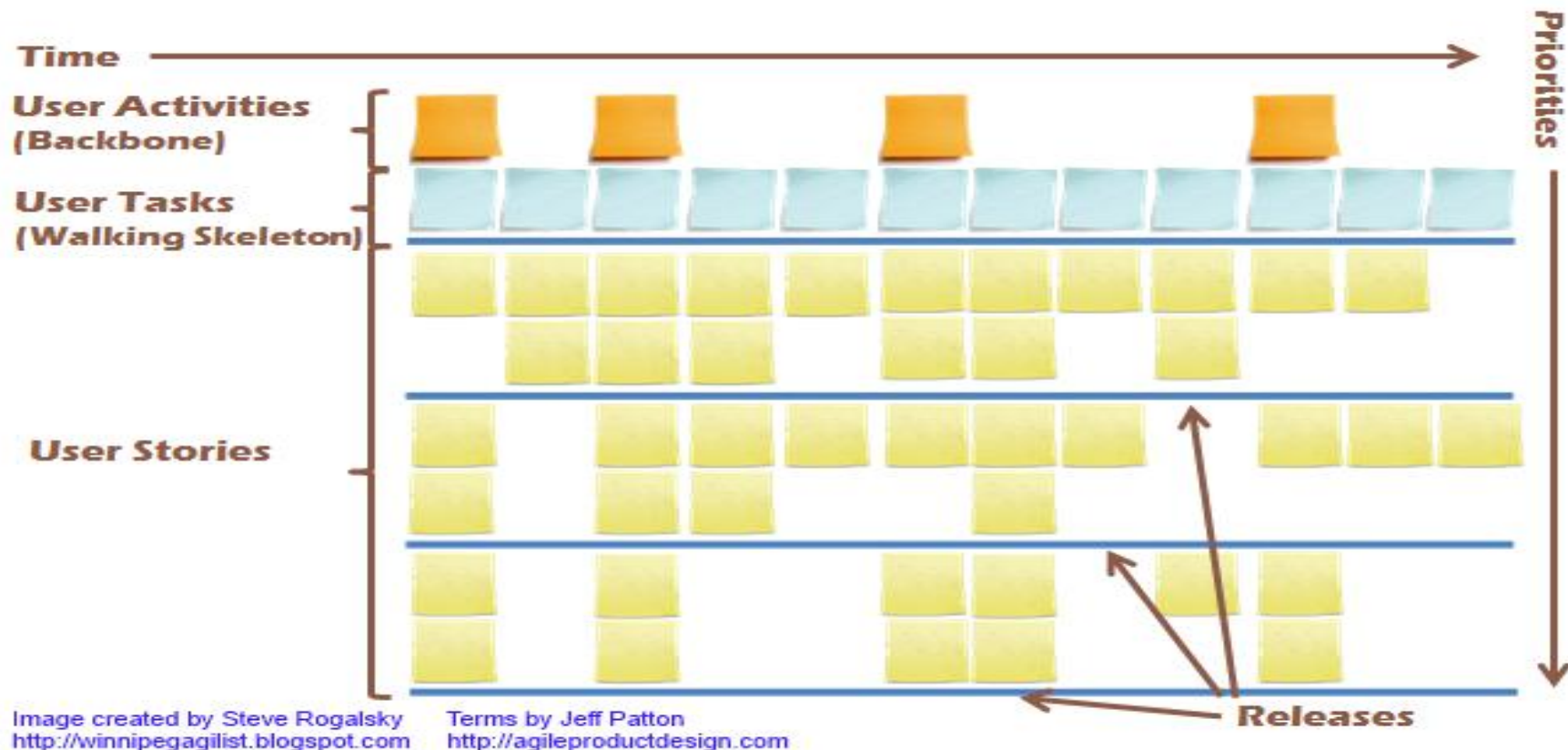
- 定义：发布计划是一个高层级计划，通常覆盖3~9个月，帮助客户和敏捷团队决定每个项目的时间范围或者每个阶段内的工作内容。
- **发布计划随着开发的进行不断调整**
 - 工作内容
 - 时间
- **注意点：**
 - 先规划几个迭代——不做过多预言性工作
 - 不涉及资源调整，只关注实际资源

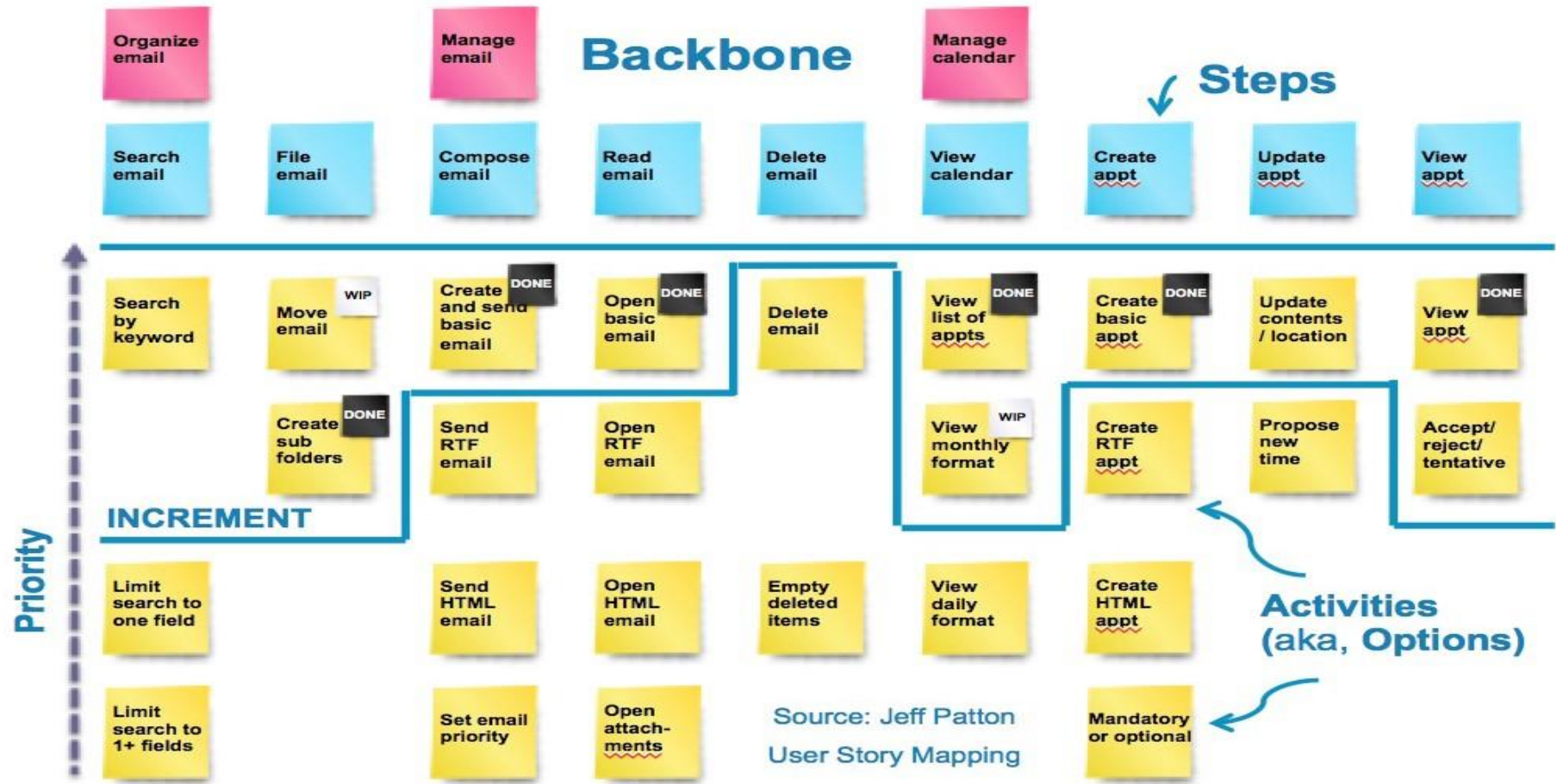
敏捷计划层次-发布计划

制定步骤：

1. 确定满足条件：由PO决定，一般与范围、进度、资源、**时间**有关
2. 初步估算：PO只需要维护开始的2-3个迭代的需求清单，不用做过多的估算
3. 选择迭代周期
4. 估算速率
5. 确定用户故事优先级
6. 选择故事和发布时间

敏捷计划层次-发布计划-用户故事地图





敏捷计划层次-迭代计划

- 定义：与发布计划很类似，但时间更短，可以看作是发布计划在某个迭代中的细化。
- 迭代计划聚焦于：本次迭代的故事的详细任务以及任务下发
- 计划方式：
 - 基于迭代速度
 - 基于团队承诺

敏捷计划层次-迭代计划-基于迭代速度

1. 调整用户故事优先级：Product Backlog
2. 预测迭代速度
3. 识别迭代目标
4. 选择用户故事——与迭代目标一致
5. 拆分用户故事为任务
6. 估算任务

敏捷计划层次-迭代计划-基于团队承诺

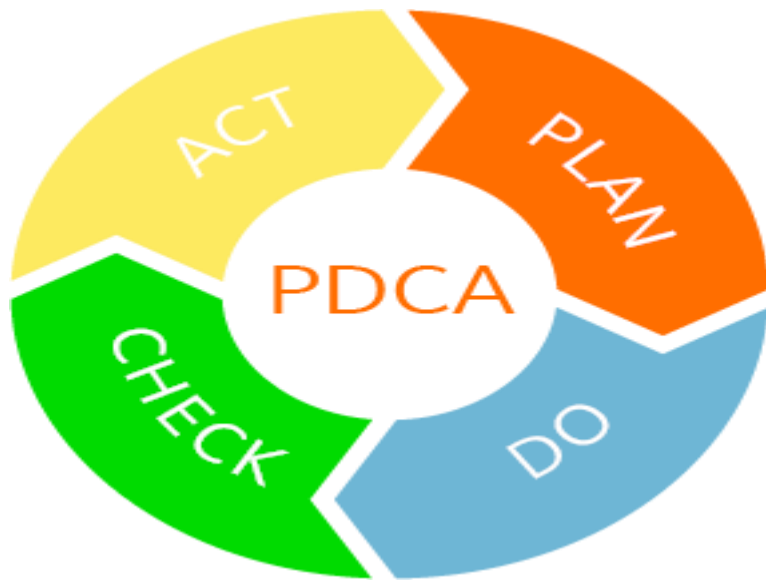
- 调整用户故事优先级：Product Backlog
- 识别迭代目标
- 通过优先级，针对某个User Story 进行承诺，在本迭代中团队是否可以完成，若可以完成，则将User Story纳入本次迭代计划；否则更换一个更小的User Story重复上述工作；
- 重复上述工作，直到团队不可以再承诺任何User Story

敏捷计划的工具

- 时间盒
- 渐进明细
- 最小可售功能 (MMF)
- 最小可用产品 (MVP)
- 基于价值的分析
 - 定义愿景

持续改进

持续改进本质



持续改进-回顾

- 定义：回顾是迭代之后的一个经验总结教训会，团队聚在一起检查并改进团队绩效。
- 优点：
 - 改进可以在当前项目即时生效
- 类型：
 - 改进生产力
 - 改进能力
 - 改进质量
 - 改进技能

持续改进-回顾步骤

- 定基调

- ☐ 确定数据和参与者
- ☐ 回顾流程方法和讨论主题，也就是目标
- ☐ 明确价值观以及回顾工作原则
- ☐ 让人们用正确的情绪提供信息和想法

- 定基调的活动

- ☐ 签到
- ☐ ESVP：人员分类
 - ☐ Explorers，探索者，想学到新的东西
 - ☐ Shoppers，采购者，愿意带着新东西回去
 - ☐ Vacationers，度假者，来休息的人，没有实际价值
 - ☐ Prisoners，囚徒，被迫来参加会议的人

持续改进-回顾步骤

- 收集数据：见前面内容
- 分析原因：见前面内容
- 采取行动：见前面内容
- 回顾收尾
 - 明确改进点
 - 对改进建议进行评价
 - 有帮助
 - 有阻碍
 - ETC
 - 表达感谢

Scrum

The Agile - Scrum Framework

Inputs from Executives,
Team, Stakeholders,
Customers, Users



Product Owner



The Team



Product Backlog

Team selects starting at top as much as it can commit to deliver by end of Sprint

Sprint Planning Meeting



Sprint Backlog



Scrum Master



Burndown/up Charts

Every 24 Hours



Daily Scrum Meeting

1-4 Week Sprint

Sprint end date and team deliverable do not change



Sprint Review



Finished Work



Sprint Retrospective

Scrum

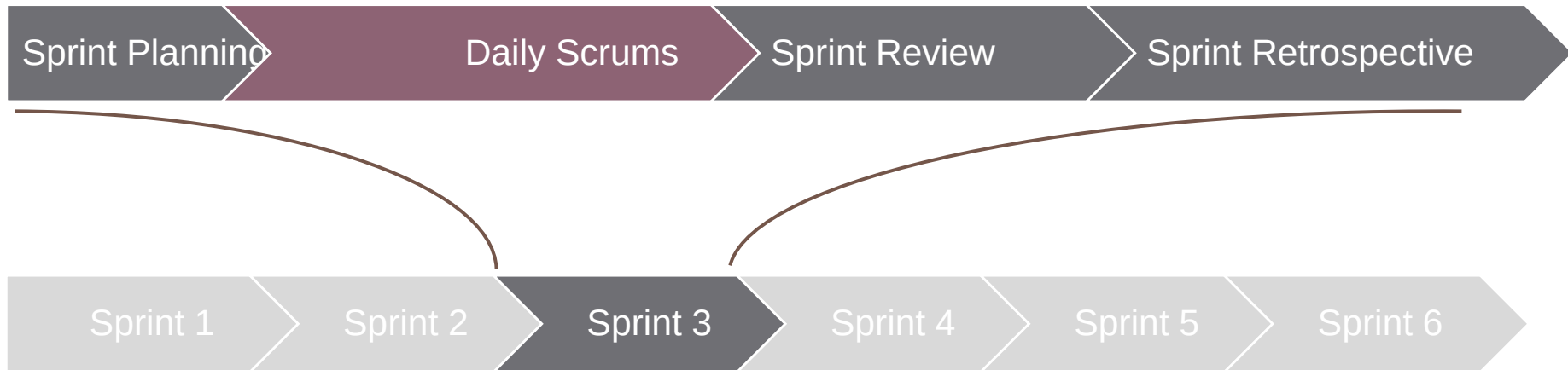
开发模式

Sprint $\equiv$ Scrum 中的迭代



Scrum

Events



Scrum

事件 | Sprint

时间盒（最长1个月）



- 有最长持续时间
- **不可以**临时拓展时长
- 聚焦价值



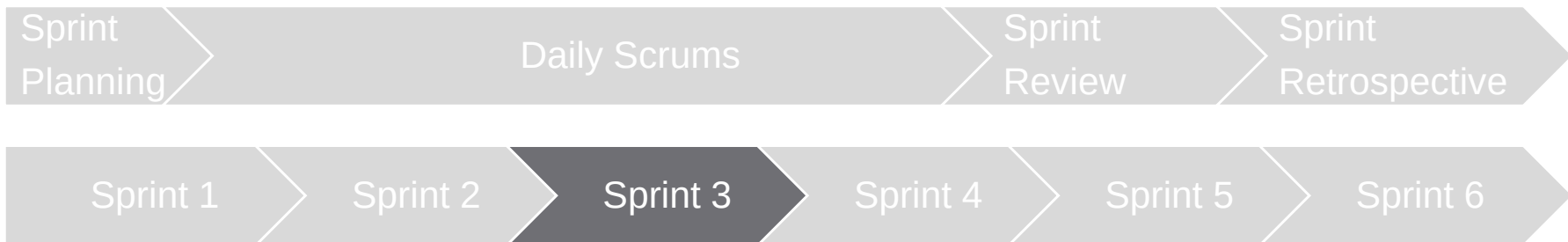
Scrum

事件 | Sprint

在 Sprint 内不可改变以下内容:

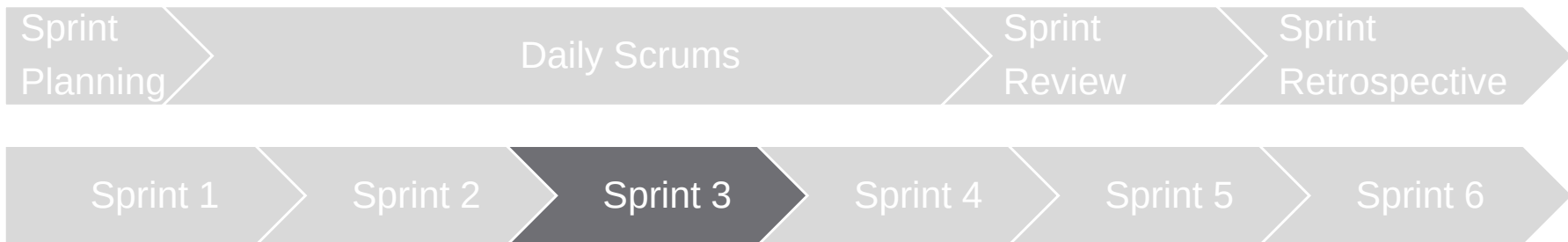
- 开发团队成员
- Sprint 计划
- 预期质量

创建高效环境



Sprint 取消

- Sprint 目标过时了
- Product Owner的权力



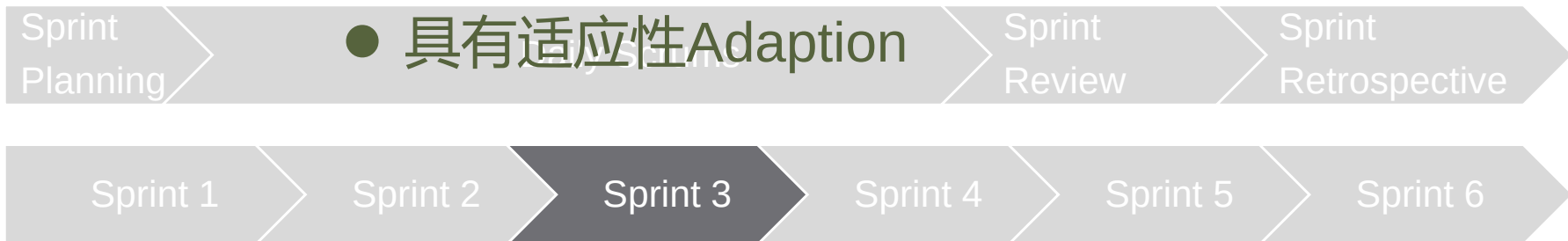
Scrum

事件 | Sprint

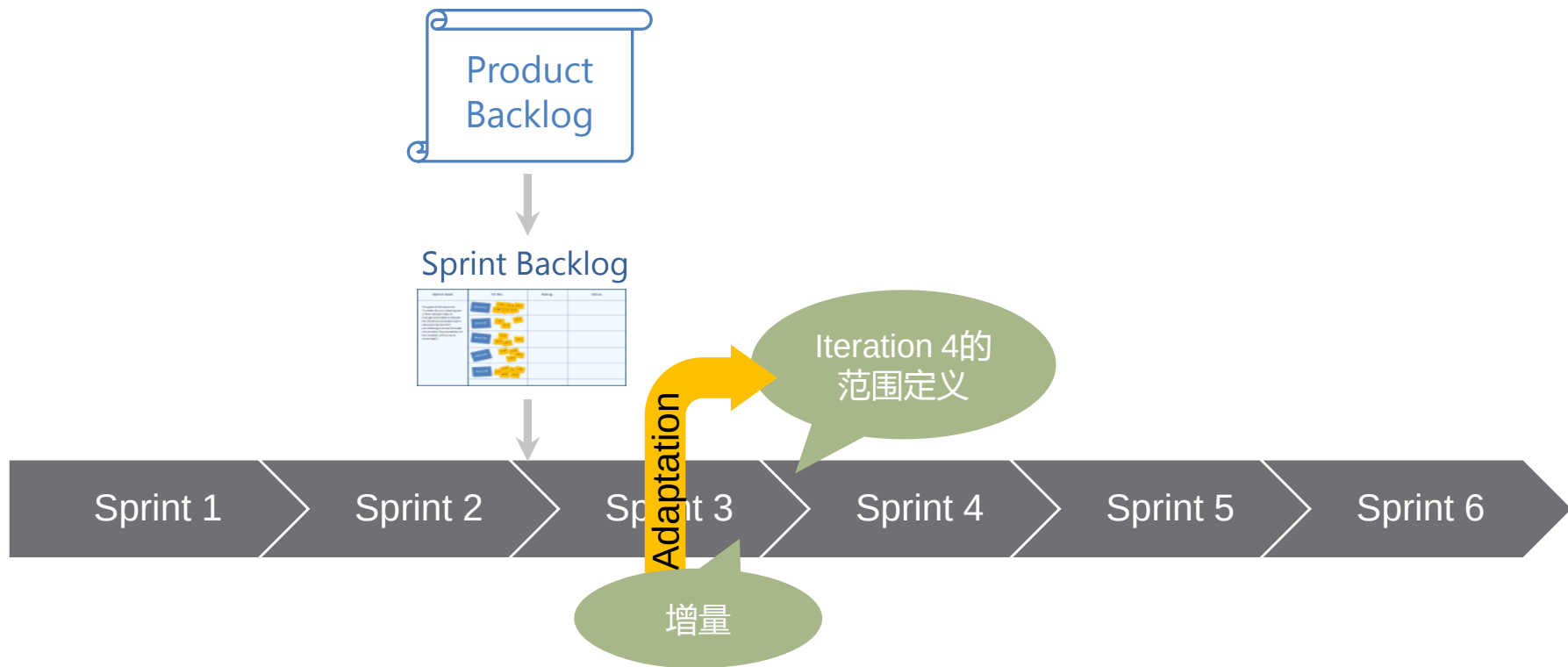
目标: 创建 增量



- 可运行的软件
- 对最终用户可用
- 潜在可发布
- 具有适应性Adaption

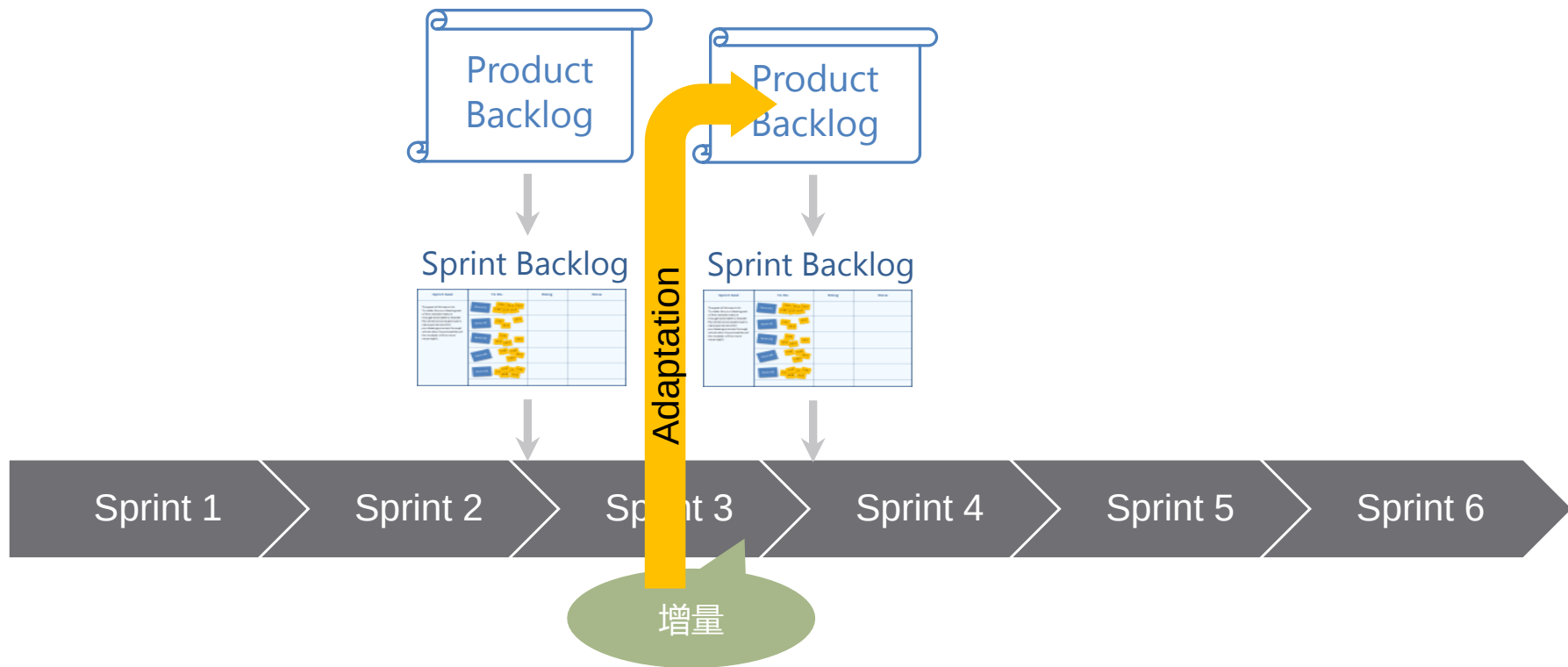


开发模式



Scrum

开发模式



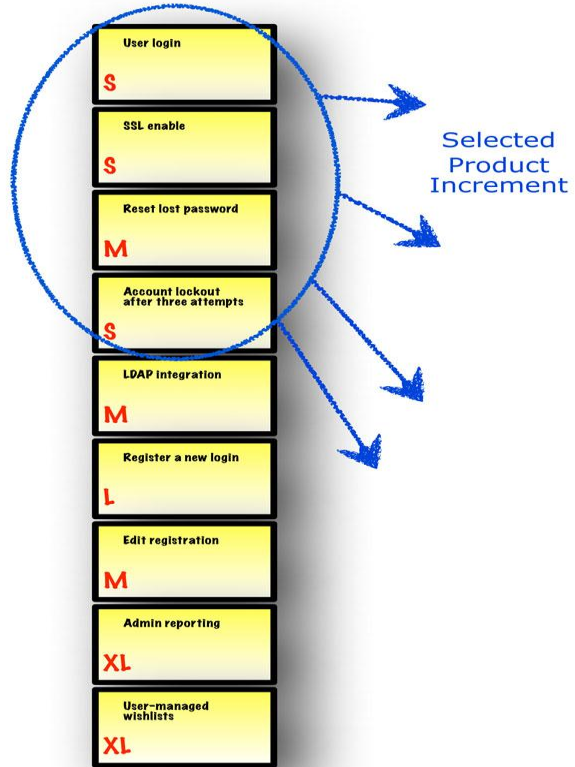
2

Backlogs

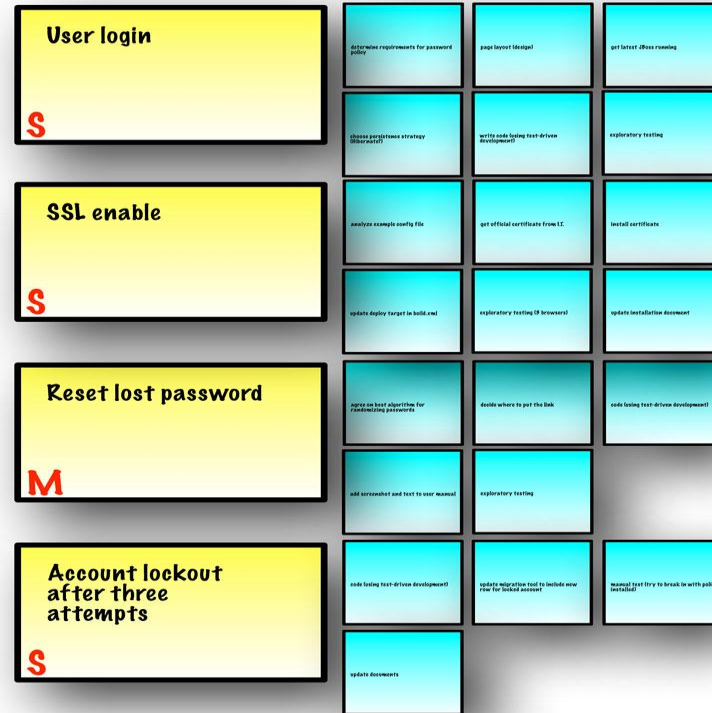
在团队内部共享“完成”的定义，达成共识。

- 开发过程
- 非功能性features
- 质量标准
- Etc.

Product Backlog



Sprint Backlog

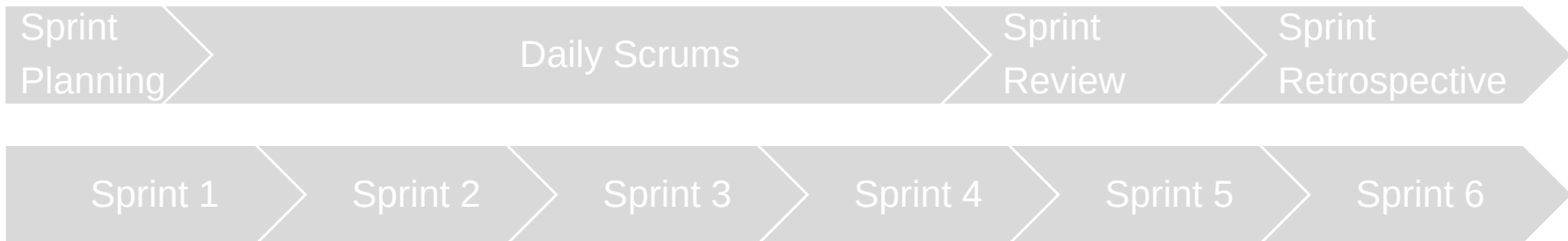


Scrum

工件 | Product Backlog产品待办项



- 排序的features列表
- 动态，永远不会完成



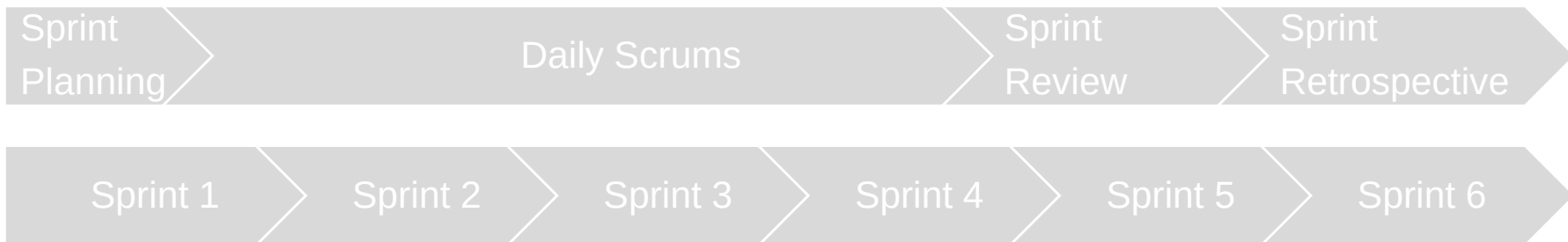
Scrum

工件 | Product Backlog产品待办项



Product Backlog:

- 非技术性
- 独立性



Scrum

工件 | Product Backlog产品待办项



INVEST

Independent

Negotiable

Valuable

Estimate-able

Small

User Story

Testable

Sprint
Planning

Daily Scrums

Sprint
Review

Sprint
Retrospective

Sprint 1

Sprint 2

Sprint 3

Sprint 4

Sprint 5

Sprint 6

Scrum

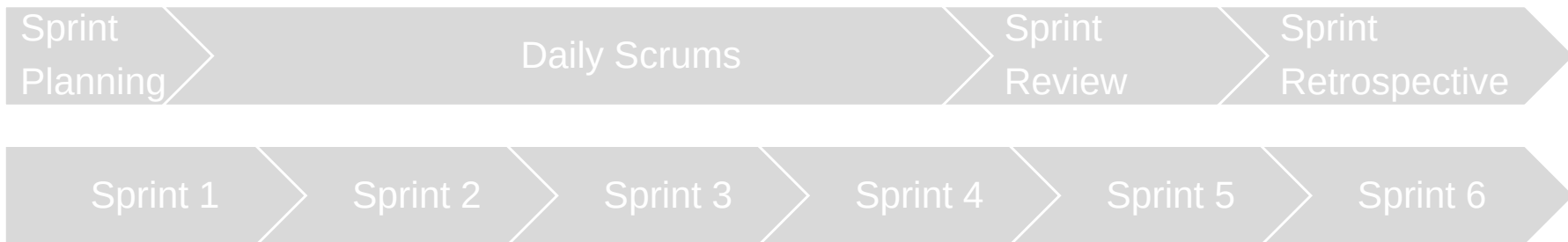
工件 | Product Backlog产品待办项



User Stories

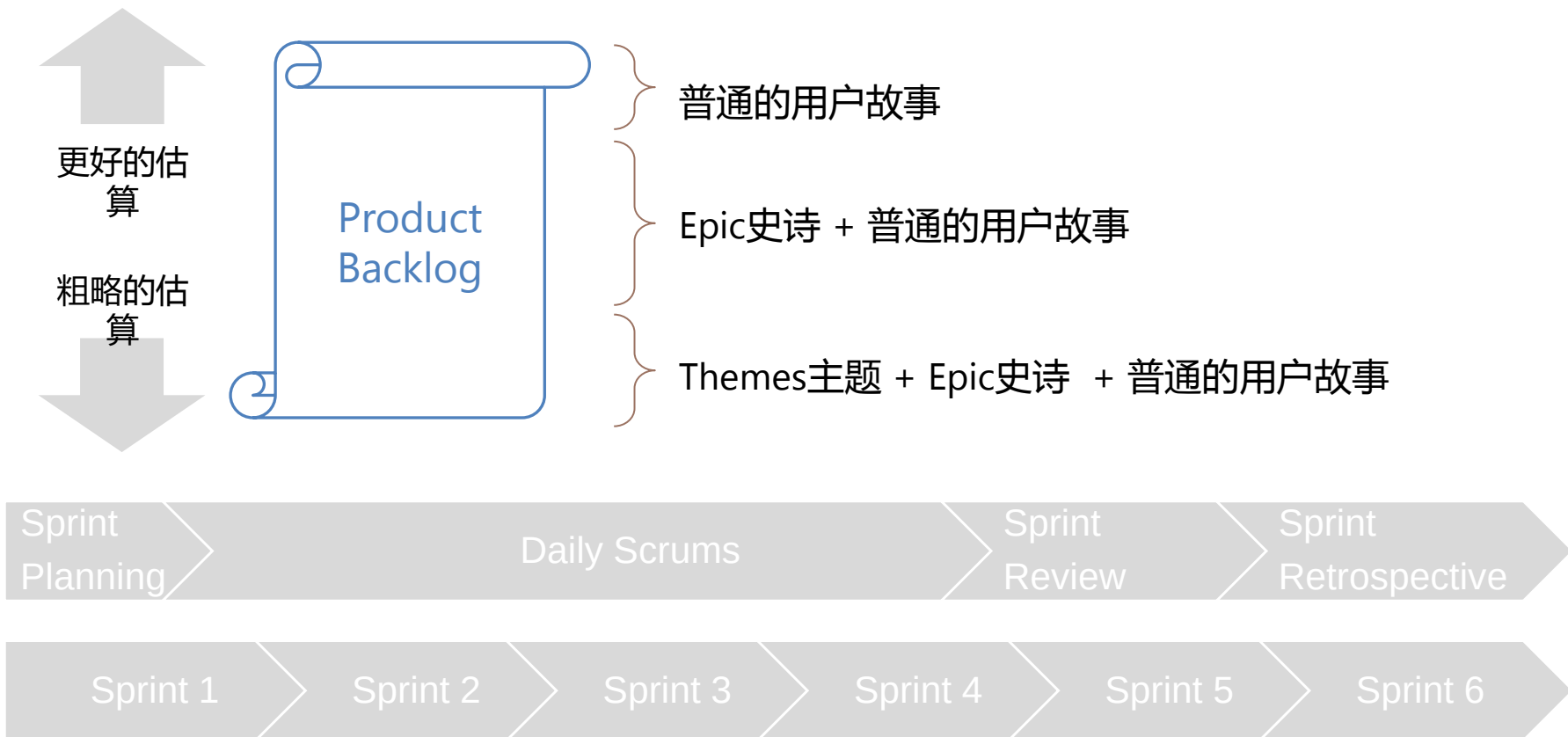
As a ..., I want to ..., [so that ...]

作为一个<角色>，我想要<功能>，
以便于<商业价值>



Scrum

工件 | Product Backlog产品待办项



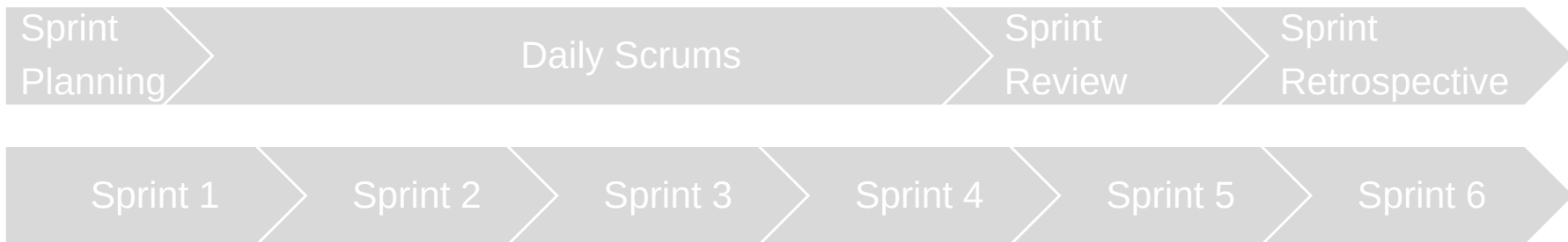
Scrum

工件 | Product Backlog产品待办项



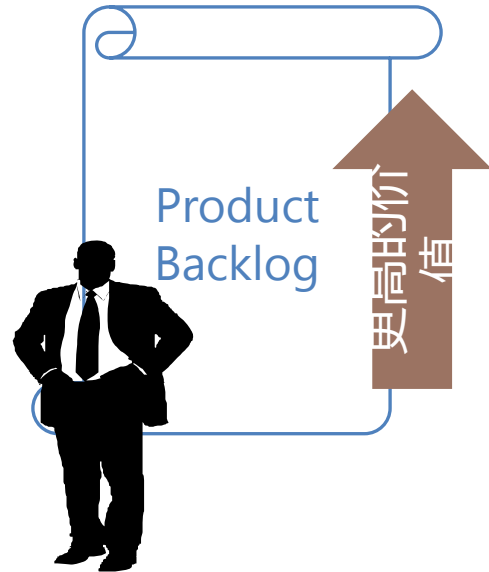
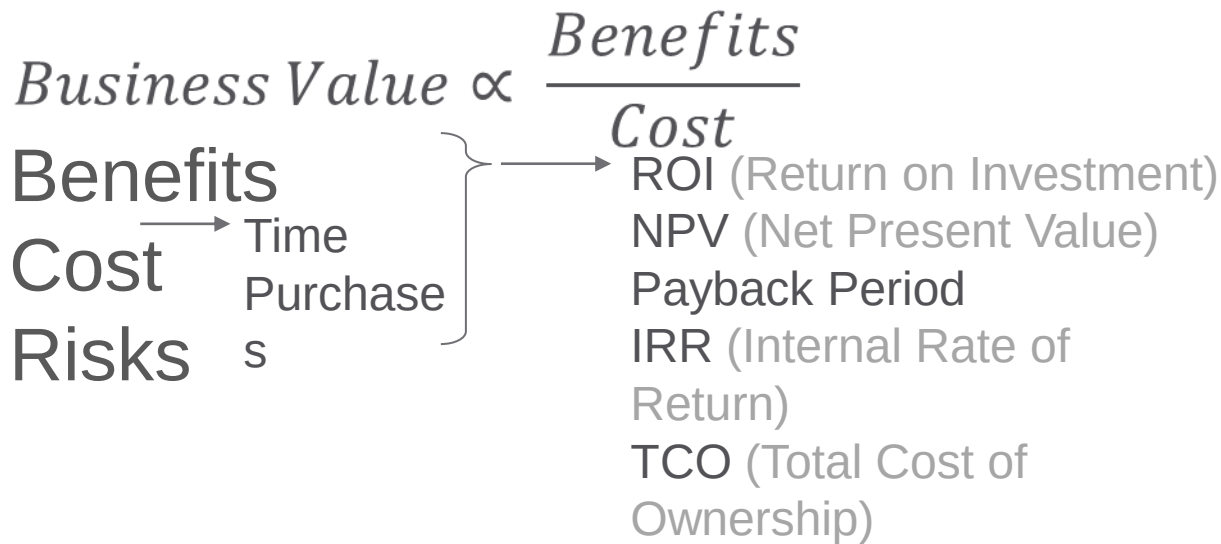
Product Owner:

- 被 Product Owner 拥有
- 创建条目 (User Story)
- 对条目进行排序
- 澄清条目



Scrum

工件 | Product Backlog 产品待办项 | 排序



Scrum

工件 | Product Backlog产品待办项

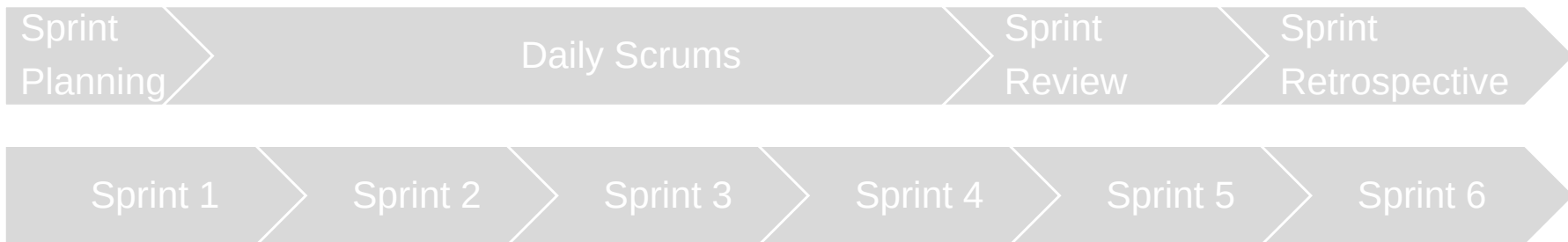


Product Backlog 优化（梳理）：

增加新的条目/更多的细节

持续性活动

不超过10%的开发时间



Scrum

工件 | Sprint Backlog Sprint待办项

Sprint Goal	To Do	Doing	Done
The goal of this sprint is To make the purchasing part of the website mature enough to be able to handle the whole process and users can experience a full purchasing process, through which other functionalities of the website will be more meaningful.	Item #1 t.1.6 t.1.3 t.1.2 t.1.4 t.1.1 t.1.5		
	Item #2 t.2.1 t.2.3 t.2.2		
	Item #3		
	Item #4		
	Item #5		



Scrum

工件 | Sprint Backlog Sprint待办项

固定的

2

固定的

1

每日更新

3

Sprint Goal The goal of this sprint is To make the purchasing part of the website mature enough to be able to handle the whole process and users can experience a full purchasing process, through which other functionalities of the website will be more	To Do	Doing	Done
	Item #1		
	Item #2		
	Item #3		
	Item #4		
	Item #5		



Scrum

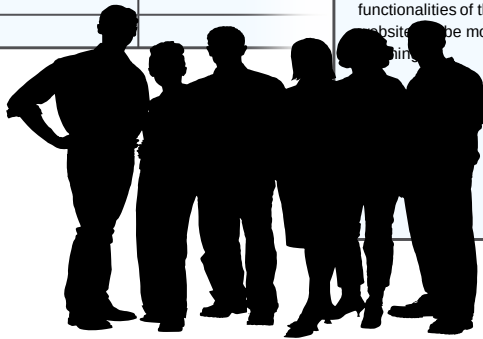
工件 | Sprint Backlog Sprint待办项

Sprint Planning (计划会议) 结束后

Sprint Goal	To Do	Doing	Done
The goal of this sprint is To make the purchasing part of the website mature enough to be able to handle the whole process and users can experience a full purchasing process, through which other functionalities of the website will be more meaningful.	Item #1 t.1.6 t.1.3 t.1.2 t.1.4 t.1.1 t.1.5		
	Item #2 t.2.1 t.2.2 t.2.3		
	Item #3		
	Item #4		
	Item #5		

几天后

Sprint Goal	To Do	Doing	Done
The goal of this sprint is To make the purchasing part of the website mature enough to be able to handle the whole process and users can experience a full purchasing process, through which other functionalities of the website will be more meaningful.			Item #1 t.1.6 t.1.1 t.1.3 t.1.4 t.1.5 t.1.2
	Item #2 t.2.7	t.2.6 t.2.5	t.2.1 t.2.3 t.2.2 t.2.4
	Item #3 t.3.4 t.3.5 t.3.3 t.3.2	t.3.1	
	Item #4 t.4.4 t.4.2 t.4.5		
	Item #5		



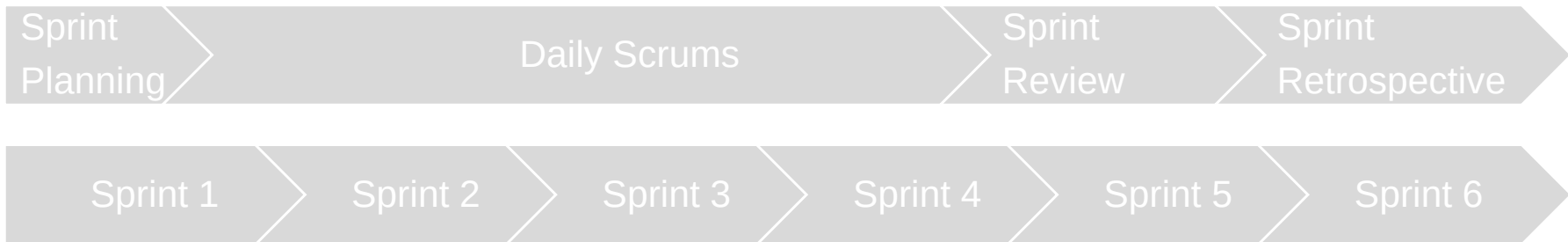
Scrum

工件 | Sprint Backlog Sprint待办项

- Sprint Backlog 由开发团队拥有
- 责任分摊
- 团队负责测量Sprint 绩效

Sprint Backlog

Sprint Backlog	Item	Estimate	Actual
Item 1	Item 1	1	
Item 2	Item 2	2	
Item 3	Item 3	3	
Item 4	Item 4	4	
Item 5	Item 5	5	
Item 6	Item 6	6	
Item 7	Item 7	7	
Item 8	Item 8	8	
Item 9	Item 9	9	
Item 10	Item 10	10	



User Story

用户故事

用户故事是由**Product Owner**编写的，描述对用户有价值的功能。

三要素

角色：谁

功能：要什么

价值：为什么要

好的用户故事



举例

1. 作为一个用户，我需要可以注册账号，这样子我就可以登录并保存我的个人信息
2. 作为卖家，我可以上传我的商品，这样子我就可以在网上出售我的商品

3种 角色

Scrum 角色

Product Owner



1 Person

全职 或者 兼职

关注于商业价值

Scrum Master



1 Person

全职 或者 兼职

Scrum expert

Development Team



3 to 9 Persons

最好是全职

technical people

Scrum

角色



团队特点：

- 自组织
- 跨职能

Scrum

角色



- 没有额外的角色
- 没有岗位名称

Scrum

工作地点

- 相同地点办公 (渗透式沟通)

- 面对面沟通

- 可变化的家具

信息发射源

分布式团队...



Scrum team

Product owner

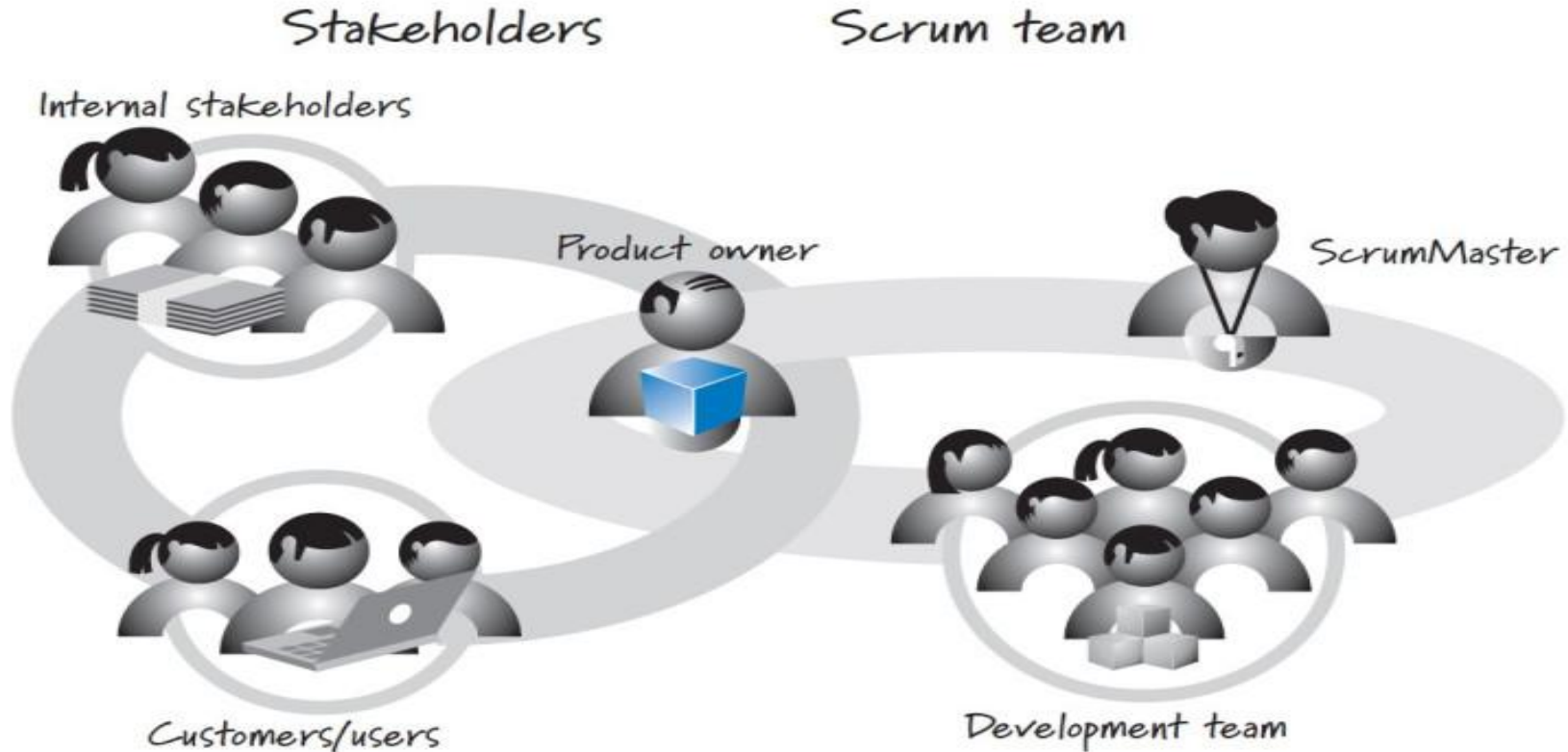


ScrumMaster



Development team

Product Owner





Scrum

Scrum

Scrum Master

- 对**过程**负责（不对解决方案负责，也不对人负责）
- 确保Scrum被正确理解及贯彻
- 教练以及问题解决
- 清除障碍
- 保护团队
- 促进各项活动开展



Scrum

Product Owner

- 对Product Backlog负责（交付价值）
- 创建产品条目
- 对产品条目进行排序
- 澄清需求



Scrum



Scrum

Development Team

- 完成Sprint Backlog
- 自组织
- 估算
- 更新信息发射源
- 报告
- ETC.....



PO

Represent Stakeholders
Manage Vision
Write requirements
Manage ROI

AL

Add to PBL
Manage PBL
Manage Release
Accepts work

SM

Maintain PBL
Decides on release dates
Inspect results

Give Feedback

Facilitate collaboration

Manage Process

Facilitate meetings

Assists Product Owner

Attend meetings

Participates in Sprint Planning

Shield team

Teaches and coaches Scrum framework

Guarantees process transparency

Remove Impediments

Available during Sprint

Planning

Improve-ment

DEV

Meets daily face to face

Identify Impediments

Demonstrates Sprint results

Manage Sprint Backlog

Follow up progress

Commit to work

Allocate tasks

Develop ways of working

Manage Sprint

Create SBL

Estimating

Self-organizes

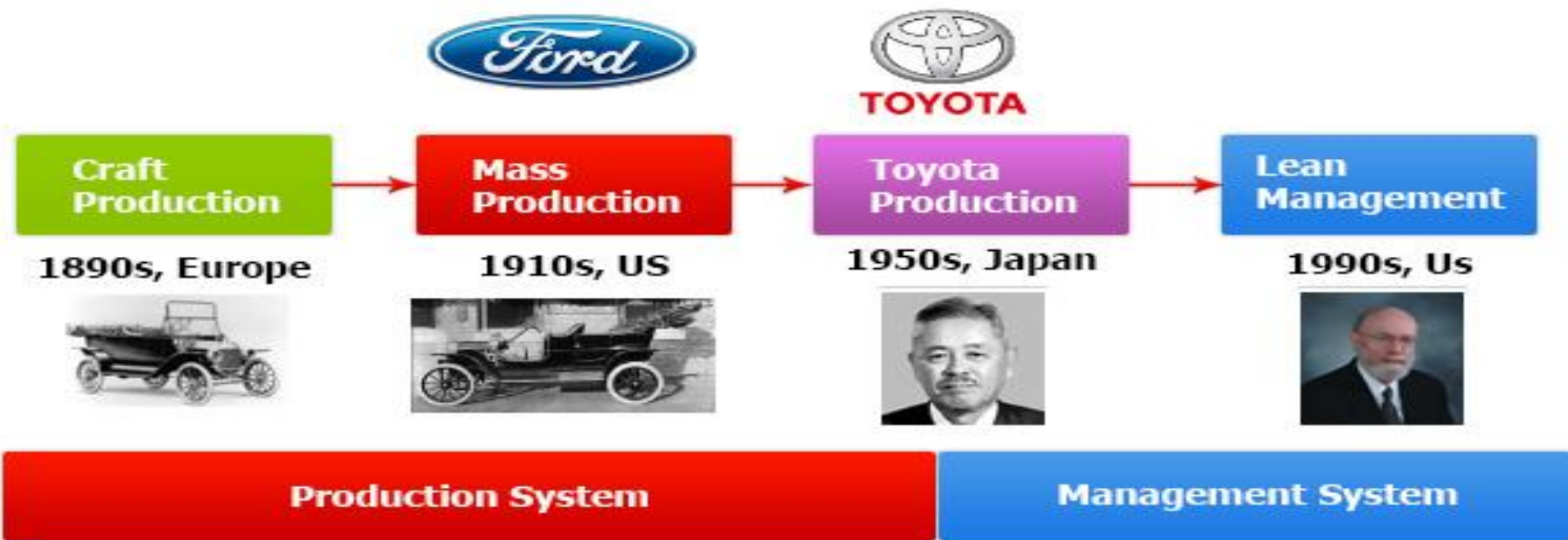
Develop product

Owens Sprint plan

Reporting

精益开发

精益的历史

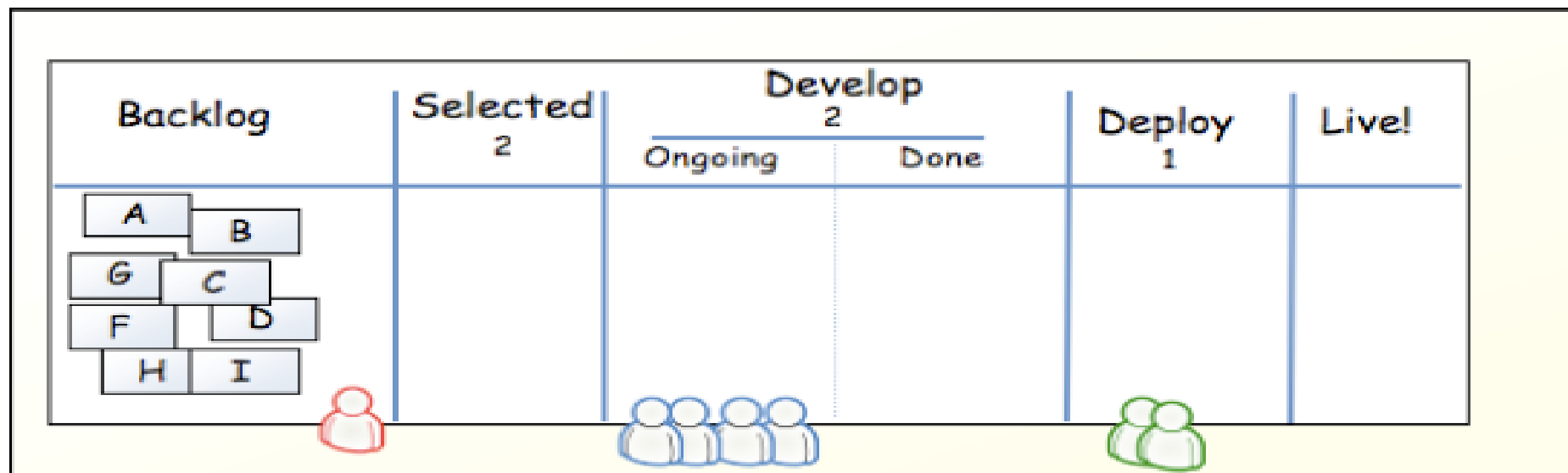


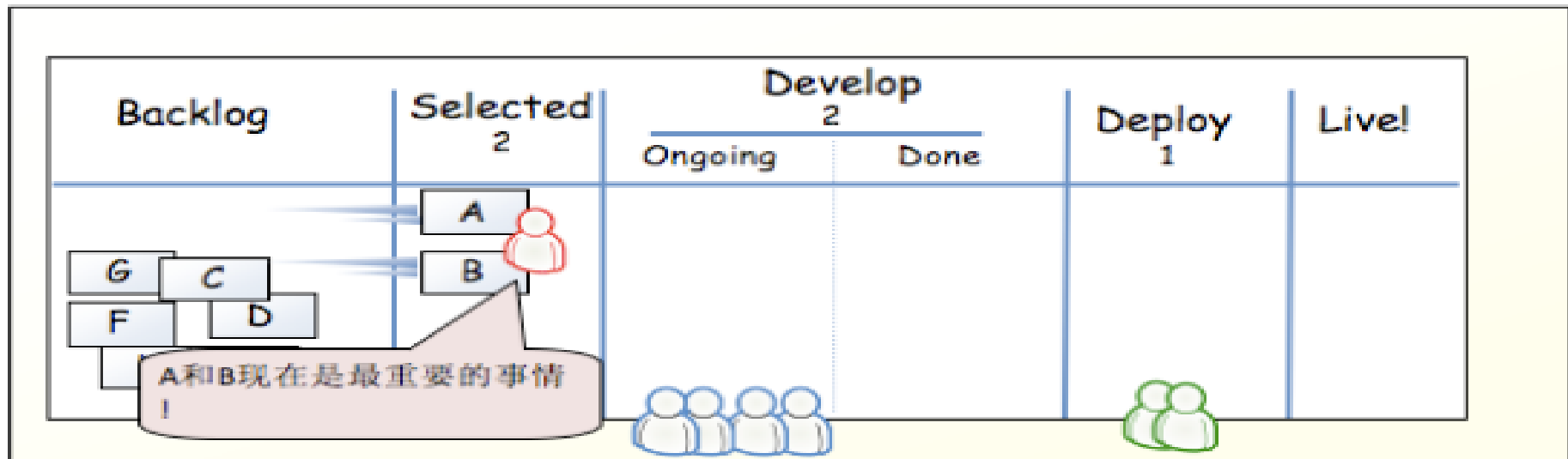
精益/Lean 开发原则

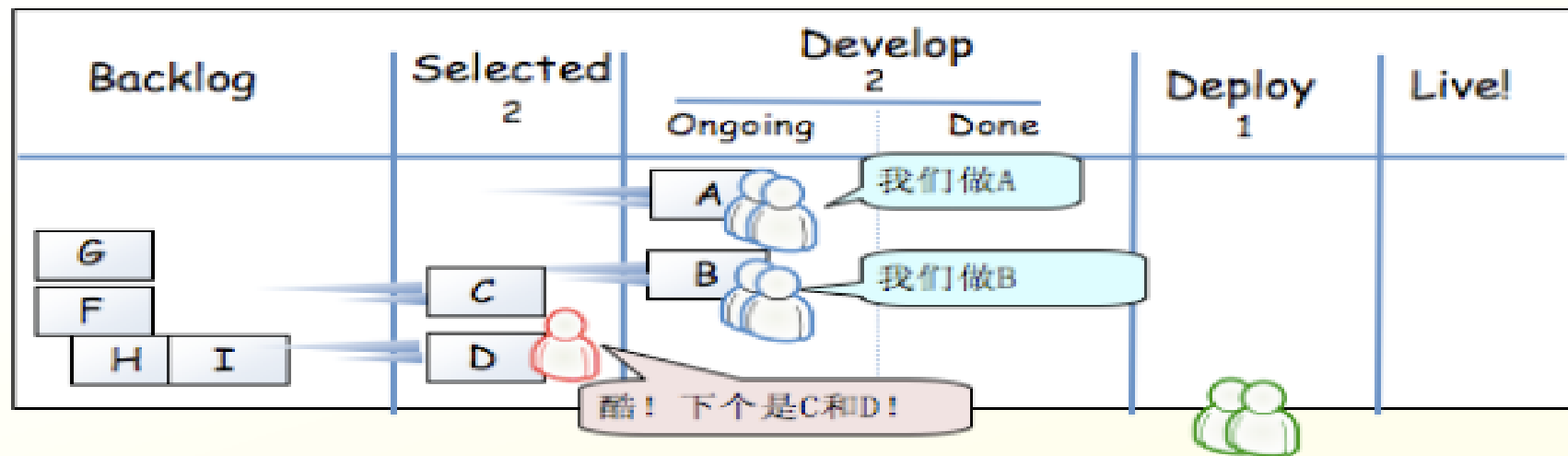
- Eliminate waste : 消除浪费
- Build Integrity(Quality) in : 构建完整性
- Amplify Learning : 强化学习
- Decide as late as possible : 尽可能晚做决策
- Deliver as fast as possible : 尽可能早交付
- Respect People(Empower Team) : 尊重授权团队
- Optimize the Whole : 全盘优化

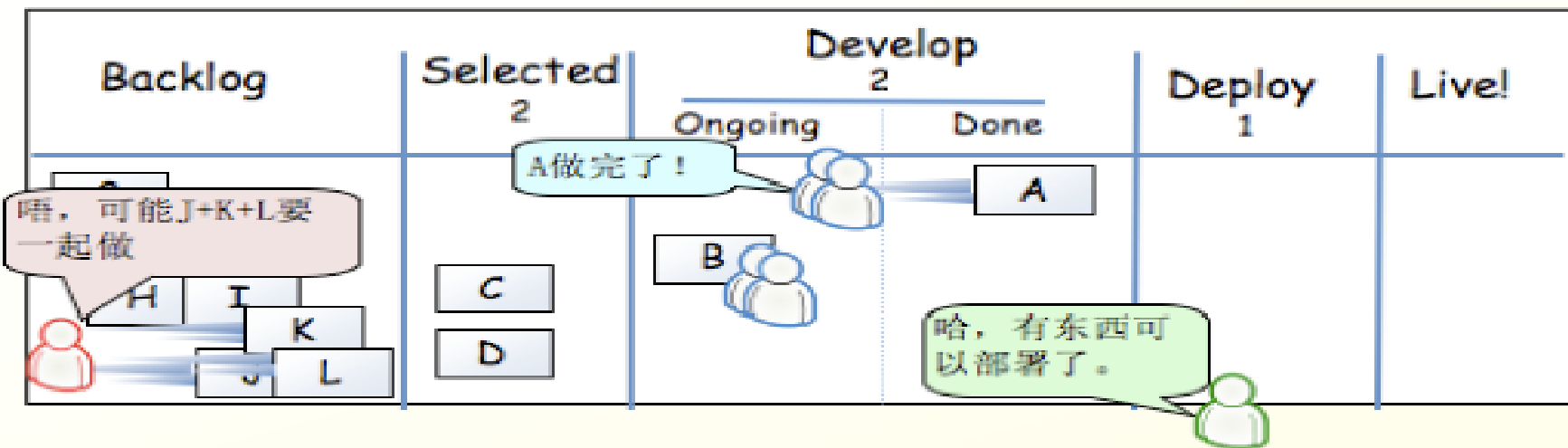
KANBAN 的原则

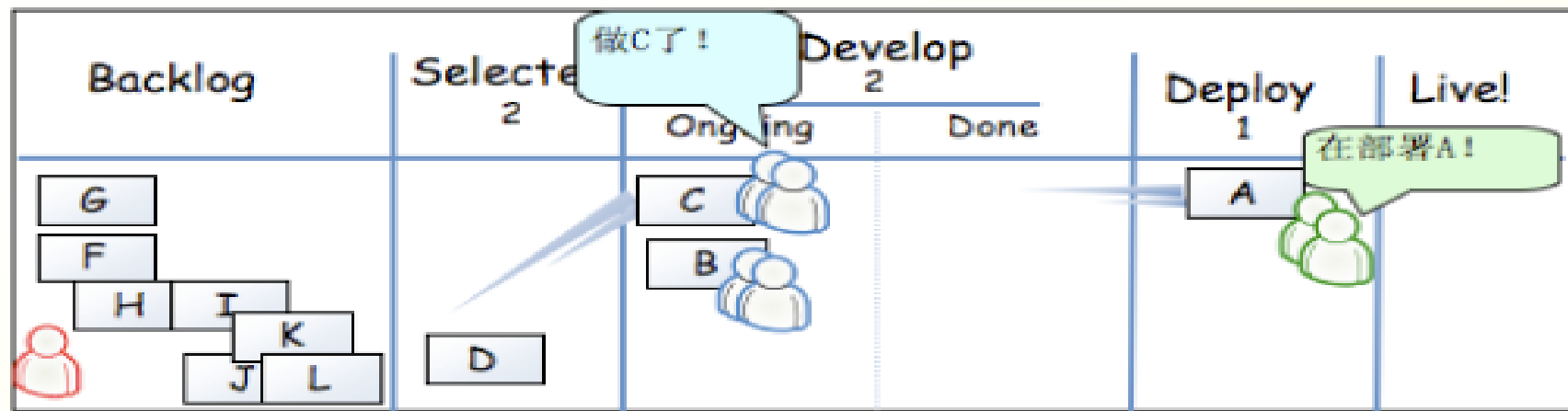
- Visualize the Workflow : 可视化工作流
- LIMIT WIP : 限制在制品数量
- Manage Flow(Pull) : 管理工作流
- Make Process Policies Explicit : 保证流程规范清晰明确
- Improve Collaboratively : 加强合作

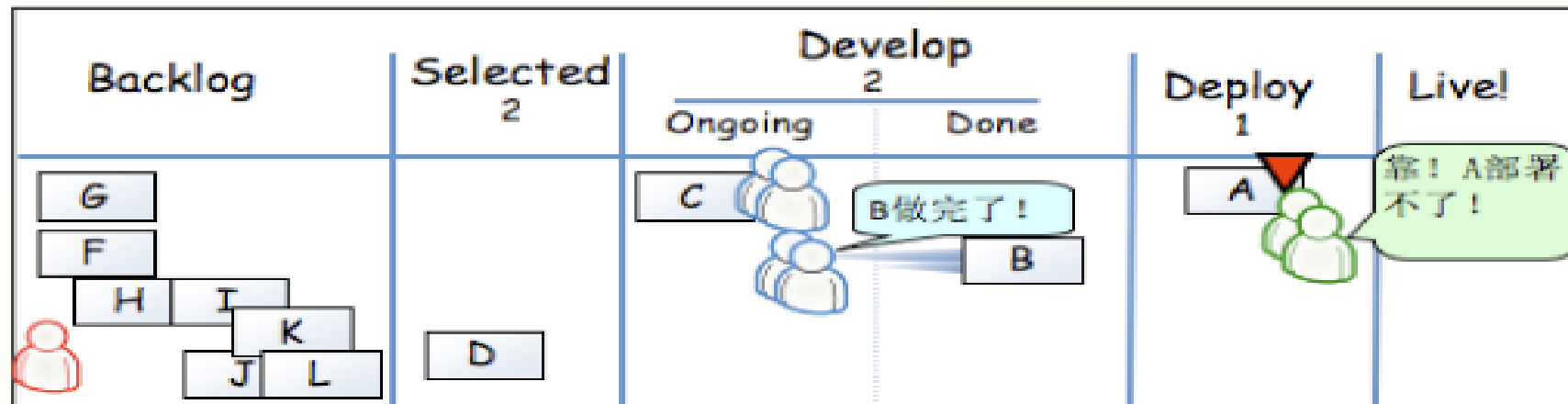


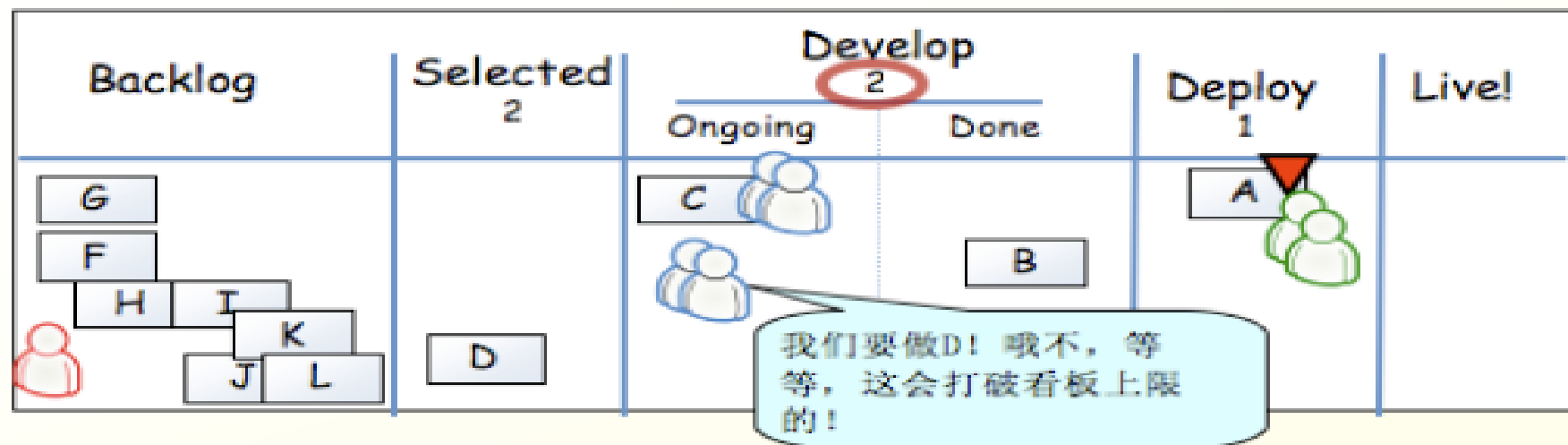


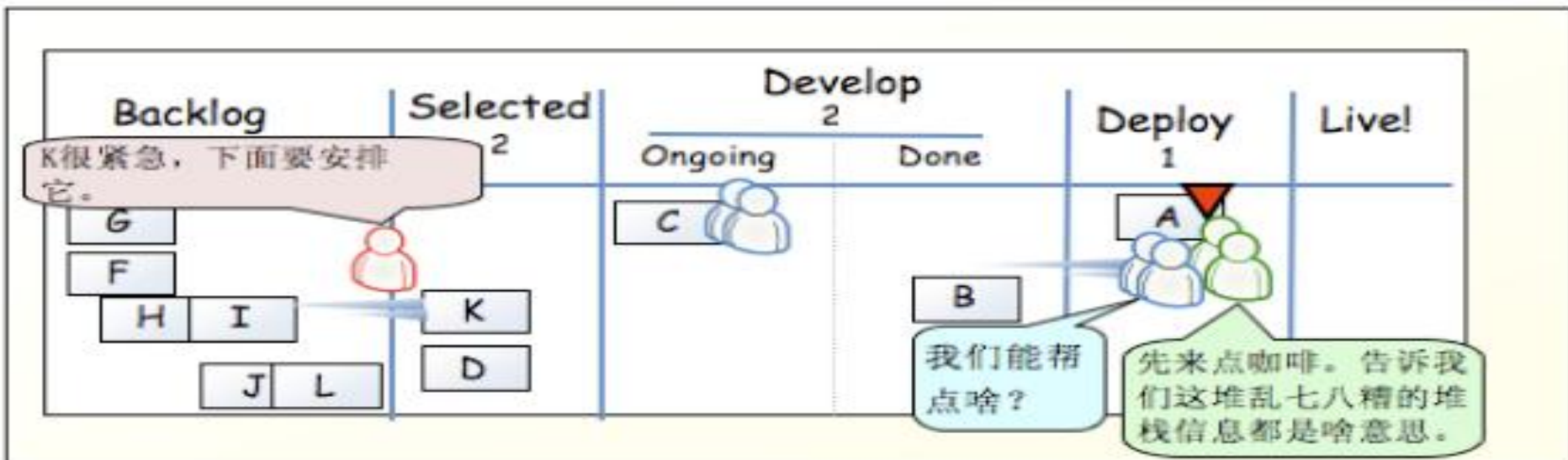


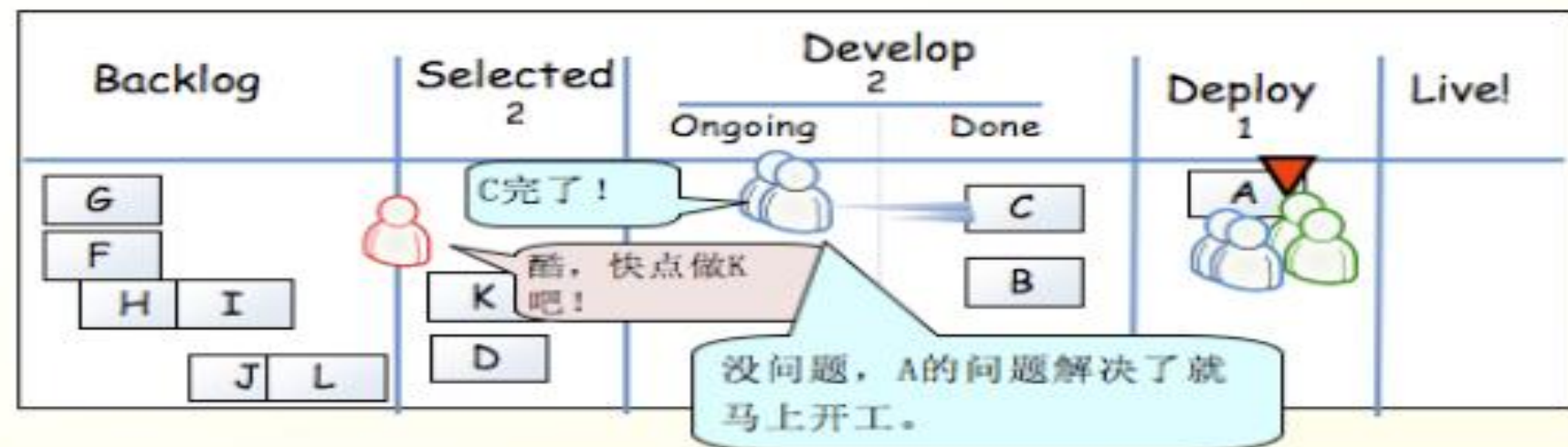


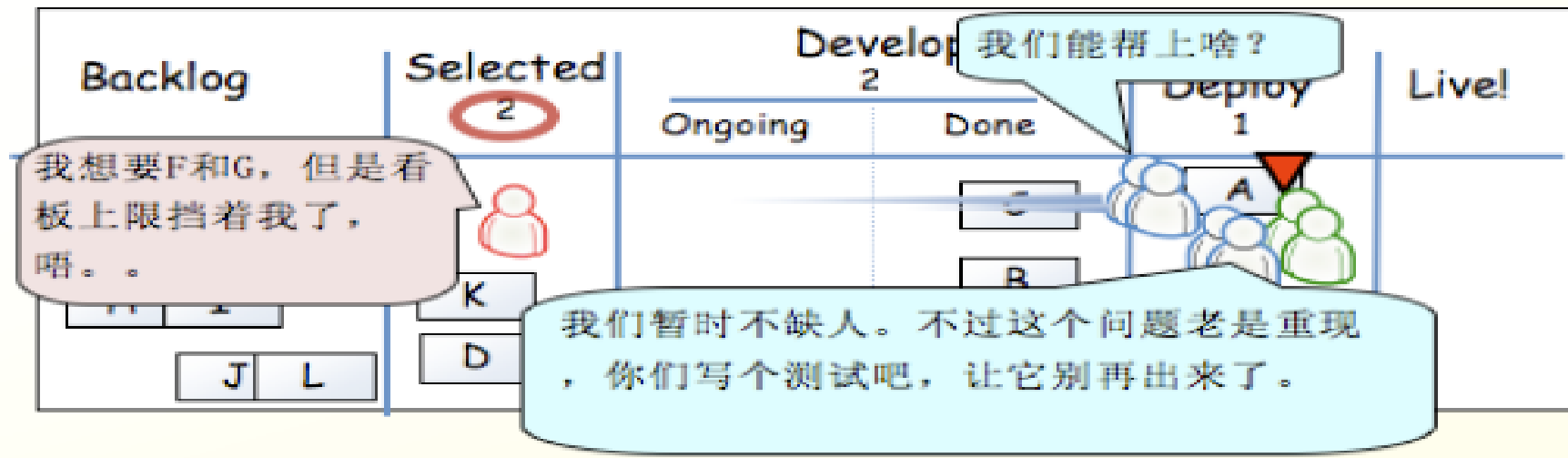


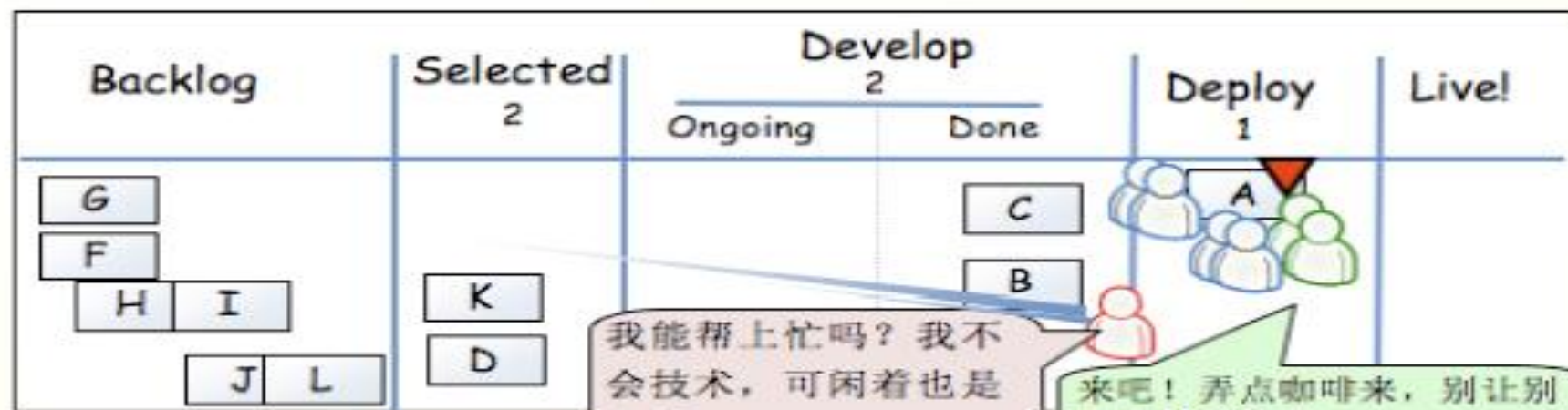












几天之后...

Backlog

Selected
2

Develop
3

Ongoing

Done

Deploy
1

Live!

H M
J L
R

I

G
F

D

K

A
B
C

M还是H? 当然是M! 不,
H! 嗯...不, 还是M! M还
是H还是M还是H? ?? 啊啊啊

你们还记得那些日子么? 还
记得通宵搞集成, 大砣大砣
的发布么?

哈哈

Scrum VS Kanban

Scrum

- 设置时间盒
- 团队预先选择每个冲刺的工作
- Cross-Function Team
- 间接限定WIP（时间盒）
- 必须要有估算（不用精确）
- 每个Sprint 中不能改变工作
- 一个Sprint Backlog 由一个研发Team 负责
- Scrum Board会在每个冲刺之间重置

Kanban

- 时间盒是可选的
- 预先选择工作是可选的
- 除了Cross-function，专家团队也是可选的
- 直接限制WIP
- 估算是可选的
- 随时可以增加工作量，只要容量允许（WIP限定）
- 一个Kanban可以由多个Team分享
- Kanban Board是可选的

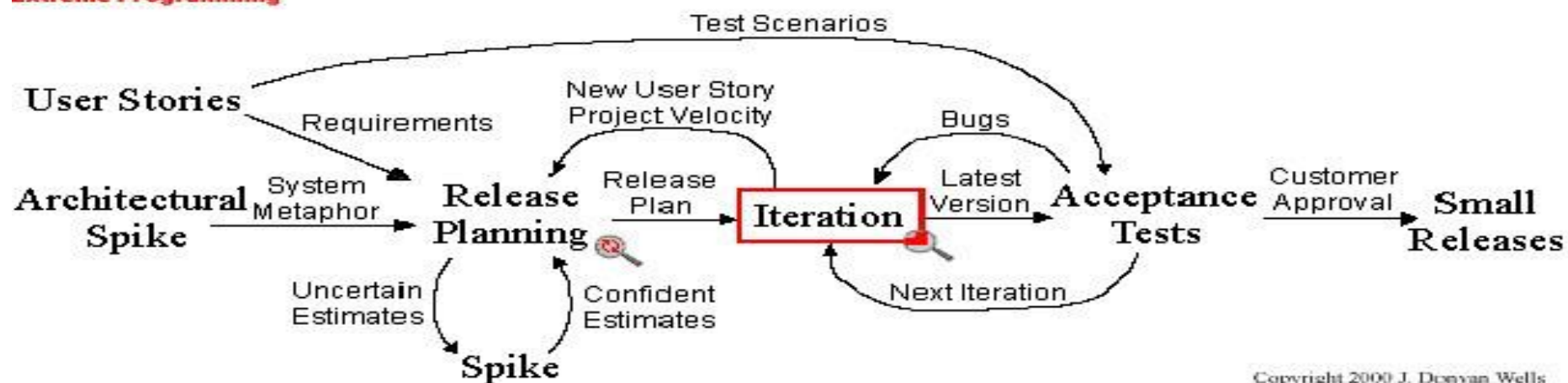
极限编程

EXtreme Programming

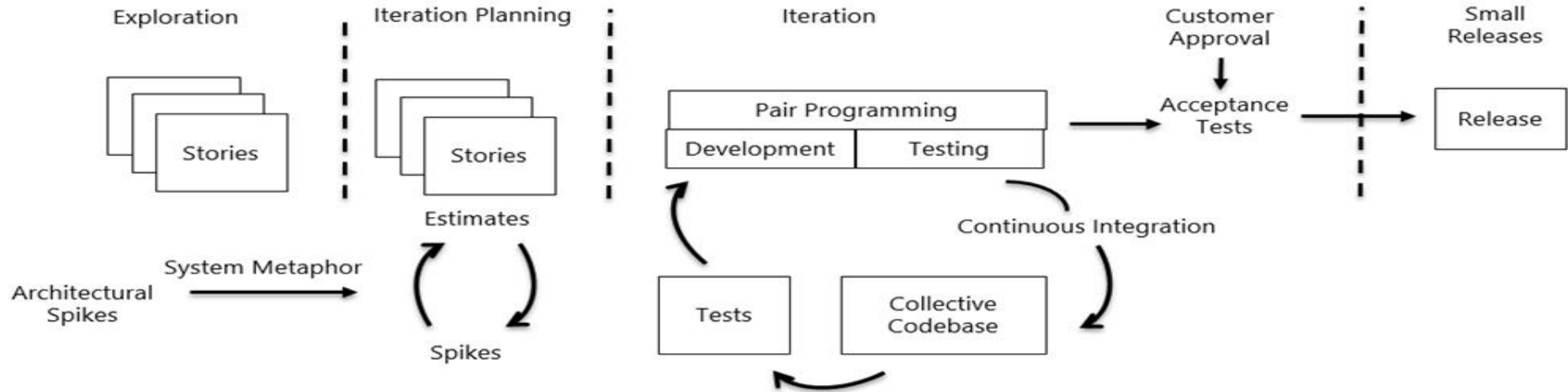
- XP，我们一般称为极限编程，是最轻量级的开发流程。
- 最主要的精神是：『在客户有系统需求时，给予及时满意的可执行程序』，所以最适合需求快速变动的项目。
- 它强调：客户所要的是workable的执行代码，所以把与撰写程序无关的工作降至最低，并要求客户与开发人员最好以side-by-side的方式一起工作

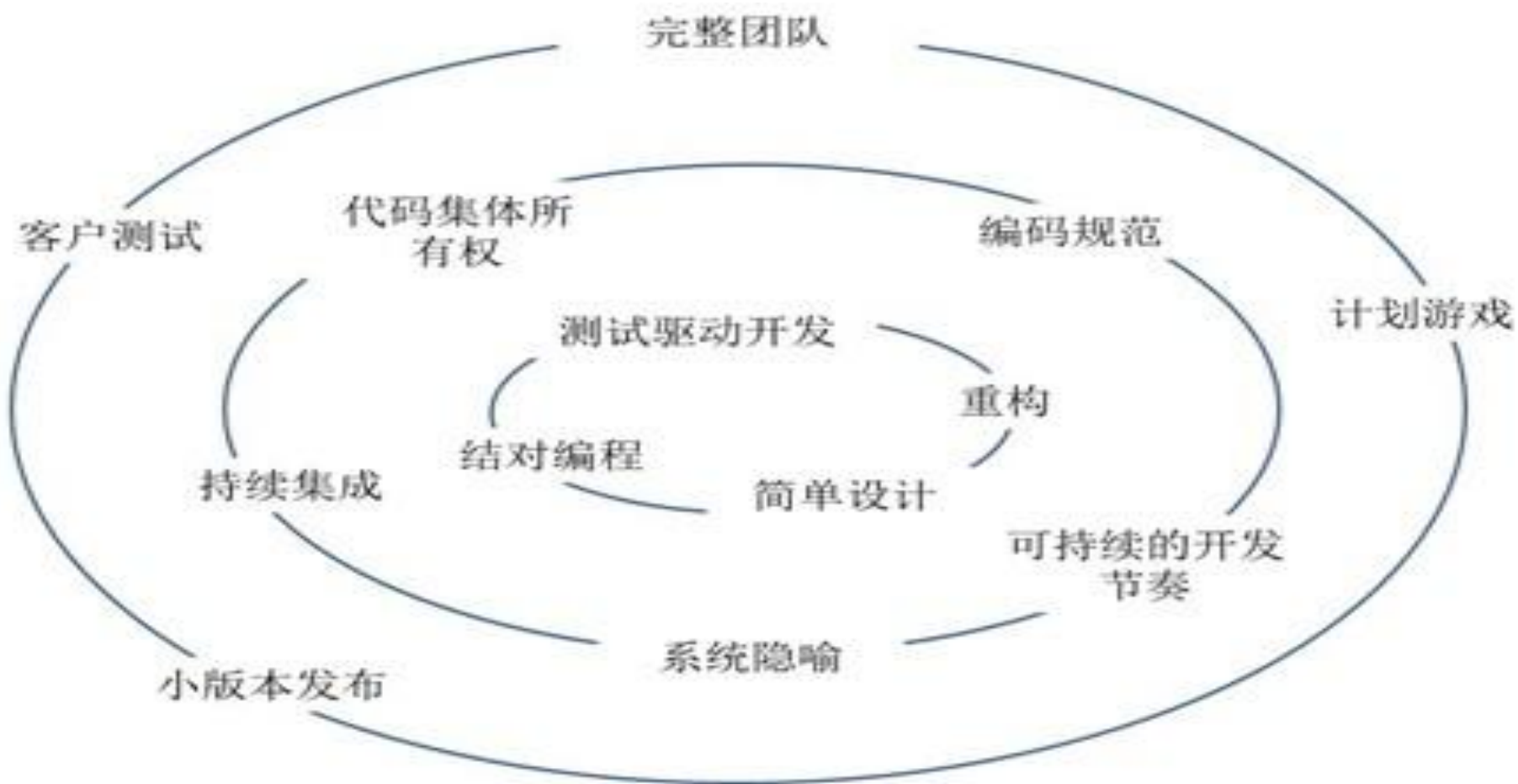


Extreme Programming Project



Extreme Programming (XP) at a Glance





XP的价值观

- Simplicity
- Communication
- Feedback
- Courage
- Respect

XP的12个实践

- Planning Game 计划游戏
- Small Releases 小版本发布
- Metaphor 隐喻
- TDD 测试驱动开发
- Refactoring 重构
- Pair Programming 结对编程
- Collective Code Ownership 代码集体所有
- Continuous Integration 持续集成
- Sustainable Pace 可持续的开发速度
- Customer Tests 客户测试
- Code Standards 编码规范

XP的角色

- XP Coach - XP教练
- XP Customer – XP客户
- XP Programmer – XP程序员
- XP Tester – XP测试员
- XP Tracker – XP跟踪员